

SitiaHome

Σύστημα Εύρεσης Κατοικίας με χρήση AI



Νικόλαος Μπελιμπασάκης
Μηχανικός Η/Υ και Πληροφορικής

Περιεχόμενα

1. Περίληψη του Project	2
2. Διαδικασία Ανάπτυξης	4
2.1 Συγκέντρωση και Ανάλυση Απαιτήσεων	4
2.2 Σχεδιασμός Συστήματος	8
2.2.1 Αρχιτεκτονική Υψηλού Επιπέδου	9
2.2.2 Ροή Δεδομένων	12
2.2.3 Σχεδιασμός Δομικών Στοιχείων	14
2.2.4 Επιλογή Τεχνολογιών	26
2.3 Υλοποίηση	29
2.3.1 Εκτέλεση και Αρχικοποίηση της Εφαρμογής	31
2.3.2 Ανάλυση Κώδικα και Ροής Εκτέλεσης	35
2.3.3 Παραδείγματα Εκτέλεσης και Συμπεριφοράς της Εφαρμογής	55
3. Συμπεράσματα και Μελλοντικές Επεκτάσεις	66
3.1 Συμπεράσματα	66
3.2 Μελλοντικές Επεκτάσεις	67
4. Αναφορές	69

1. Περίληψη του Project

Σκοπός και Λειτουργικότητα

Το SitiaHome είναι ένα Software/AI Engineering project που έχει ως στόχο την ανάπτυξη μιας εφαρμογής για την υποστήριξη χρηστών που αναζητούν κατοικία προς ενοικίαση στον Δήμο Σητείας, Κρήτης.

Η εφαρμογή θα δέχεται σε φυσική γλώσσα τις ανάγκες και τις προτιμήσεις του χρήστη, θα αναζητά διαθέσιμες αγγελίες στην πλατφόρμα Χρυσή Ευκαιρία (ΧΕ) και θα αξιολογεί τις επιλογές που εντοπίζει, ώστε να παρουσιάζει τις κατοικίες που ταιριάζουν καλύτερα στα ζητούμενα του χρήστη.

Το project υλοποιείται πρωτίστως για εκπαιδευτικούς και portfolio σκοπούς, με έμφαση στην εφαρμογή μιας οργανωμένης Software Engineering διαδικασίας και στην αξιοποίηση σύγχρονων AI τεχνολογιών όπου αυτές προσφέρουν ουσιαστική αξία. Παρότι δεν σχεδιάζεται αρχικά ως εμπορικό προϊόν ή για άμεσο launch στο ευρύ κοινό, η δομή και η λειτουργικότητά του μπορούν να αποτελέσουν τη βάση για ένα πραγματικό real-world scenario, στο οποίο ένα αντίστοιχο σύστημα θα μπορούσε να χρησιμοποιηθεί για την αυτοματοποίηση και τη διευκόλυνση της διαδικασίας εύρεσης κατοικίας.

Μοντέλο Ανάπτυξης Λογισμικού

Για την ανάπτυξη του SitiaHome ακολουθείται μια διαδικασία βασισμένη στο Software Development Life Cycle (SDLC) [1]. Σύμφωνα με την IBM, το SDLC αποτελεί μια δομημένη και επαναληπτική μεθοδολογία για την ανάπτυξη, παράδοση και συντήρηση λογισμικού και περιλαμβάνει επτά βασικές φάσεις: Planning, Analysis, Design, Coding, Testing, Deployment και Maintenance. Κάθε φάση έχει διαφορετικούς στόχους και παραδοτέα και λειτουργεί ως βάση για την επόμενη. Στο SitiaHome ακολουθείται αυτή η γενική δομή, προσαρμοσμένη όμως στις ανάγκες και στον χαρακτήρα του project. Επειδή πρόκειται για ένα ατομικό, εκπαιδευτικό και portfolio project και όχι για εμπορικό προϊόν που αναπτύσσεται από οργανωμένη development team, ορισμένες δραστηριότητες των επιμέρους φάσεων δεν εφαρμόζονται με την ίδια έκταση που θα είχαν σε ένα πραγματικό επιχειρησιακό περιβάλλον.

Ως μοντέλο ανάπτυξης επιλέγεται μια Iterative προσέγγιση. Στο Iterative model δημιουργείται αρχικά μια μικρή λειτουργική έκδοση του λογισμικού, συχνά ένα Minimum Viable Product (MVP), και στη συνέχεια το σύστημα βελτιώνεται μέσα από διαδοχικές επαναλήψεις. Για το SitiaHome αυτό σημαίνει ότι αρχικά υλοποιήθηκε η βασική λειτουργική ροή της εφαρμογής και στη συνέχεια πραγματοποιήθηκαν σταδιακές διορθώσεις και βελτιώσεις στα επιμέρους components, καθώς προέκυπταν νέες ανάγκες ή προβλήματα κατά την ανάπτυξη.

Η επιλογή αυτής της προσέγγισης δεν σημαίνει ότι αγνοήθηκε ο αρχικός σχεδιασμός. Αντίθετα, πριν από την υλοποίηση πραγματοποιήθηκαν οργανωμένα τα στάδια Requirements Analysis και Design, ώστε να υπάρχει σαφής εικόνα της δομής και της λειτουργίας του συστήματος. Παράλληλα, η διαδικασία δεν αντιμετωπίστηκε ως αυστηρά γραμμική. Κατά την υλοποίηση, το Testing πραγματοποιήθηκε κυρίως μέσα από επαναλαμβανόμενους κύκλους εκτέλεσης, εντοπισμού προβλημάτων και διόρθωσης του κώδικα. Μετά την ολοκλήρωση της βασικής υλοποίησης, η εφαρμογή δοκιμάστηκε επιπλέον με διαφορετικά αιτήματα χρηστών, ώστε να εξεταστεί η συμπεριφορά της σε διαφορετικά σενάρια αναζήτησης, validation, ανάκτησης αγγελιών και παραγωγής προτάσεων.

Οι φάσεις Deployment και Maintenance δεν αποτελούν μέρος του scope της παρούσας έκδοσης του project. Το SitiaHome αναπτύχθηκε ως MVP για εκπαιδευτικούς και portfolio σκοπούς και δεν υπήρχε στόχος παραγωγικής διάθεσης προς πραγματικούς χρήστες ή μακροχρόνιας λειτουργίας σε production περιβάλλον. Η βασική σκοπιμότητα του project ήταν η ανάδειξη και πρακτική εφαρμογή τεχνολογιών και τεχνικών όπως η χρήση Large Language Models, η εξαγωγή και επεξεργασία δεδομένων από εξωτερικές πηγές και η υλοποίηση μιας εφαρμογής αποτελούμενης από διαφορετικά components που συνεργάζονται μεταξύ τους για την ολοκλήρωση μιας ενιαίας ροής λειτουργίας.

2. Διαδικασία Ανάπτυξης

2.1 Συγκέντρωση και Ανάλυση Απαιτήσεων

Δεδομένου ότι το SitiaHome αποτελεί ένα ατομικό, εκπαιδευτικό και portfolio project και όχι ένα εμπορικό έργο που αναπτύσσεται από οργανωμένη ομάδα για πραγματικό πελάτη ή επιχείρηση, τα δύο πρώτα στάδια του SDLC, **Planning** και **Analysis**, αντιμετωπίστηκαν στην πράξη ως ένα ενιαίο στάδιο. Αρχικά διαμορφώθηκε η γενική εικόνα της εφαρμογής, προσδιορίζοντας το πρόβλημα που επιχειρεί να αντιμετωπίσει, τον σκοπό του έργου, τους βασικούς χρήστες και τον τρόπο με τον οποίο αυτοί αναμένεται να αλληλεπιδρούν με το σύστημα. Στη συνέχεια, η αρχική αυτή ιδέα αναλύθηκε σε μεγαλύτερο βάθος, με τον καθορισμό του scope, των βασικών λειτουργικών και μη λειτουργικών απαιτήσεων, και της γενικής ροής λειτουργίας της εφαρμογής. Με αυτόν τον τρόπο, τα στάδια Planning και Analysis συγχωνεύτηκαν σε μία κοινή ενότητα «Συγκέντρωση και Ανάλυση Απαιτήσεων», η οποία παρέχει τόσο τη συνολική κατεύθυνση του project όσο και μια πρώτη σαφή αποτύπωση του τι πρέπει να κάνει το σύστημα πριν περάσουμε στο αναλυτικότερο στάδιο του Design.

Πρόβλημα

Η εύρεση κατάλληλης κατοικίας για μακροχρόνια ενοικίαση αποτελεί συχνά μια χρονοβόρα διαδικασία. Ο ενδιαφερόμενος καλείται να εξετάσει μεγάλο αριθμό αγγελιών, να ελέγξει βασικά χαρακτηριστικά, όπως το κόστος ενοικίασης και τον αριθμό υπνοδωματίων, και στη συνέχεια να συγκρίνει τις διαθέσιμες επιλογές με βάση τις προσωπικές του ανάγκες και προτιμήσεις. Η διαδικασία αυτή απαιτεί επαναλαμβανόμενη αναζήτηση και χειροκίνητη αξιολόγηση των αποτελεσμάτων, γεγονός που αυξάνει τον χρόνο και την προσπάθεια που απαιτούνται για τον εντοπισμό μιας κατάλληλης κατοικίας.

Επομένως, υπάρχει χρησιμότητα σε ένα εργαλείο που θα αυτοματοποιεί μέρος της διαδικασίας αναζήτησης και αξιολόγησης των αγγελιών, επιτρέποντας στον χρήστη να εκφράζει τις απαιτήσεις του σε φυσική γλώσσα και να λαμβάνει μια μικρότερη και πιο σχετική ομάδα προτεινόμενων κατοικιών.

Σκοπός της εφαρμογής

Σκοπός της εφαρμογής είναι να διευκολύνει τη διαδικασία εύρεσης κατοικίας για μακροχρόνια ενοικίαση στον Δήμο Σητείας, αυτοματοποιώντας την αναζήτηση, το φιλτράρισμα και την αξιολόγηση διαθέσιμων αγγελιών. Ο χρήστης θα περιγράφει σε φυσική γλώσσα τις βασικές απαιτήσεις και τις πρόσθετες προτιμήσεις του, όπως το επιθυμητό εύρος ενοικίου, τον ελάχιστο αριθμό υπνοδωματίων, την περιοχή, το μέγεθος της κατοικίας και άλλα χαρακτηριστικά που θεωρεί σημαντικά. Η εφαρμογή θα αναζητά σχετικές αγγελίες στη Χρυσή Ευκαιρία (ΧΕ), θα αποκλείει όσες δεν ικανοποιούν τα βασικά κριτήρια και στη συνέχεια θα αξιολογεί τις υπόλοιπες με βάση τις συνολικές προτιμήσεις του χρήστη.

Βασικός στόχος είναι η εφαρμογή να λειτουργεί ως σύστημα υποστήριξης της διαδικασίας επιλογής κατοικίας, περιορίζοντας την ανάγκη του χρήστη να εξετάζει και να συγκρίνει χειροκίνητα μεγάλο αριθμό αγγελιών. Αντί να παρουσιάζει απλώς όλες τις διαθέσιμες επιλογές, θα τις αξιολογεί ως προς τη συνάφειά τους με τις ανάγκες του χρήστη και θα παρουσιάζει ένα περιορισμένο σύνολο από τις καταλληλότερες προτάσεις, συνοδευόμενο από τις βασικές πληροφορίες που διευκολύνουν τη σύγκριση και την τελική επιλογή.

Χρήστες

Ο βασικός χρήστης της εφαρμογής είναι ένα άτομο που αναζητά κατοικία προς ενοικίαση στον Δήμο Σητείας και επιθυμεί να βρει επιλογές που ανταποκρίνονται στις προσωπικές του ανάγκες και προτιμήσεις. Ο χρήστης θα μπορεί να περιγράφει με φυσικό τρόπο το είδος της κατοικίας που αναζητά, αναφέροντας στοιχεία όπως την επιθυμητή περιοχή, το διαθέσιμο budget, το μέγεθος, τον ελάχιστο αριθμό υπνοδωματίων και άλλα χαρακτηριστικά που θεωρεί σημαντικά. Η εφαρμογή θα αναζητά στην πλατφόρμα ΧΕ και, με βάση την περιγραφή των απαιτήσεών του, θα εντοπίζει και θα παρουσιάζει τις καταλληλότερες διαθέσιμες κατοικίες, περιορίζοντας την ανάγκη χειροκίνητης αναζήτησης και αξιολόγησης μεγάλου αριθμού αγγελιών.

Απαιτήσεις

Στο στάδιο της ανάλυσης απαιτήσεων προσδιορίζεται με μεγαλύτερη ακρίβεια τι πρέπει να μπορεί να κάνει η εφαρμογή και ποια χαρακτηριστικά πρέπει να παρουσιάζει κατά τη λειτουργία της. Οι απαιτήσεις του SitiaHome διακρίνονται σε **λειτουργικές (Functional Requirements)** και **μη λειτουργικές (Non-Functional Requirements)**.

Functional Requirements — Λειτουργικές Απαιτήσεις

Οι λειτουργικές απαιτήσεις περιγράφουν τις βασικές λειτουργίες που πρέπει να παρέχει το σύστημα:

- Ο χρήστης να μπορεί να εισάγει σε φυσική γλώσσα την περιγραφή της κατοικίας που αναζητά.
- Η περιγραφή του χρήστη να περιλαμβάνει συγκεκριμένες υποχρεωτικές πληροφορίες, όπως το μέγιστο επιθυμητό ενοίκιο και τον ελάχιστο αριθμό υπνοδωματίων.
- Η εφαρμογή να ελέγχει εάν ο χρήστης έχει δώσει έγκυρο prompt και, σε περίπτωση μη έγκυρου ή ελλιπούς αιτήματος, να ζητά από τον χρήστη να υποβάλει νέο ή συμπληρωμένο αίτημα.
- Η εφαρμογή να αναζητά διαθέσιμες αγγελίες κατοικιών στην πλατφόρμα ΧΕ.
- Να συλλέγει και να επεξεργάζεται τις σχετικές πληροφορίες των αγγελιών που εντοπίζονται.
- Να αξιολογεί τη συνάφεια των διαθέσιμων κατοικιών με τις απαιτήσεις και τις προτιμήσεις του χρήστη.
- Να αξιολογεί τις κατάλληλες αγγελίες και, εφόσον ο αριθμός τους υπερβαίνει τις 5, να επιλέγει τις 5 καταλληλότερες με βάση τις απαιτήσεις και τις προτιμήσεις του χρήστη. Αν οι κατάλληλες αγγελίες είναι 5 ή λιγότερες, να παρουσιάζονται όλες.
- Να εμφανίζει τις βασικές πληροφορίες κάθε προτεινόμενης κατοικίας, καθώς και σύνδεσμο προς την αρχική πηγή της αγγελίας.

2. Non-Functional Requirements — Μη Λειτουργικές Απαιτήσεις

Οι μη λειτουργικές απαιτήσεις περιγράφουν τα ποιοτικά χαρακτηριστικά που πρέπει να παρουσιάζει η εφαρμογή κατά τη λειτουργία της:

- Η εφαρμογή να διαθέτει απλό, κατανοητό και εύχρηστο περιβάλλον χρήστη.

- Η διαδικασία αναζήτησης και παραγωγής προτάσεων να ολοκληρώνεται σε ικανοποιητικό χρόνο απόκρισης.
- Το σύστημα να χειρίζεται με αξιόπιστο τρόπο μη έγκυρες ή ελλιπείς εισόδους χωρίς να διακόπτεται η λειτουργία του.
- Η αρχιτεκτονική να επιτρέπει την προσθήκη νέων πηγών αγγελιών ή διαφορετικών μηχανισμών αναζήτησης στο μέλλον χωρίς σημαντικές αλλαγές στο υπόλοιπο σύστημα.
- Τα δεδομένα της τρέχουσας αναζήτησης να διατηρούνται μόνο προσωρινά στη μνήμη κατά τη διάρκεια της εκτέλεσης και να μην αποθηκεύονται μόνιμα.
- Η εφαρμογή να προστατεύει τυχόν δεδομένα που παρέχονται από τον χρήστη και να μην πραγματοποιεί περιττή μόνιμη αποθήκευσή τους.
- Το σύστημα να είναι οργανωμένο και συντηρήσιμο, ώστε τα επιμέρους components να μπορούν να τροποποιούνται ή να επεκτείνονται ανεξάρτητα όσο είναι δυνατό.

Scope

Στο Scope καθορίζονται τα όρια του project, δηλαδή ποιες λειτουργίες περιλαμβάνονται στην έκδοση της εφαρμογής που θα υλοποιηθεί και ποιες παραμένουν εκτός του συγκεκριμένου έργου. Ο καθορισμός του scope είναι σημαντικός ώστε να υπάρχει σαφής προσανατολισμός κατά την ανάπτυξη και να αποφεύγεται η ανεξέλεγκτη επέκταση της λειτουργικότητας της εφαρμογής.

Το scope του SitiaHome περιλαμβάνει την ανάπτυξη ενός συστήματος αναζήτησης και πρότασης κατοικιών προς ενοικίαση στο Δήμο Σητείας. Ο χρήστης θα περιγράφει σε φυσική γλώσσα την κατοικία που αναζητά, παρέχοντας ορισμένες υποχρεωτικές πληροφορίες, όπως το μέγιστο επιθυμητό ενοίκιο και τον ελάχιστο αριθμό υπνοδωματίων, καθώς και οποιοσδήποτε επιπλέον προτιμήσεις επιθυμεί. Το σύστημα θα ελέγχει την εγκυρότητα των δεδομένων εισόδου και, εφόσον αυτά είναι επαρκή, θα αναζητά διαθέσιμες αγγελίες από την πλατφόρμα ΧΕ, θα επεξεργάζεται τις πληροφορίες τους και θα αξιολογεί τη συνάφειά τους με τις απαιτήσεις του χρήστη. Στη συνέχεια θα παρουσιάζει τις καταλληλότερες κατοικίες ως προτάσεις, μαζί με τις βασικές πληροφορίες και την αρχική πηγή κάθε αγγελίας.

Τα δεδομένα που προκύπτουν από την αναζήτηση θα διατηρούνται προσωρινά μόνο κατά τη διάρκεια της τρέχουσας εκτέλεσης της εφαρμογής και δεν προβλέπεται μόνιμη αποθήκευση ιστορικού αναζητήσεων ή στοιχείων χρηστών.

Στην παρούσα έκδοση του project **δεν** περιλαμβάνονται η πραγματοποίηση ή διαχείριση της ενοικίασης, η επικοινωνία μεταξύ ιδιοκτητών και ενδιαφερόμενων, η διαχείριση συμβολαίων ή πληρωμών, η δημιουργία και διαχείριση αγγελιών από ιδιοκτήτες, καθώς και η ανάπτυξη συστήματος λογαριασμών χρηστών ή μόνιμου ιστορικού. Η εφαρμογή επικεντρώνεται αποκλειστικά στην αναζήτηση, αξιολόγηση και πρόταση διαθέσιμων κατοικιών με βάση τις απαιτήσεις του χρήστη.

2.2 Σχεδιασμός Συστήματος

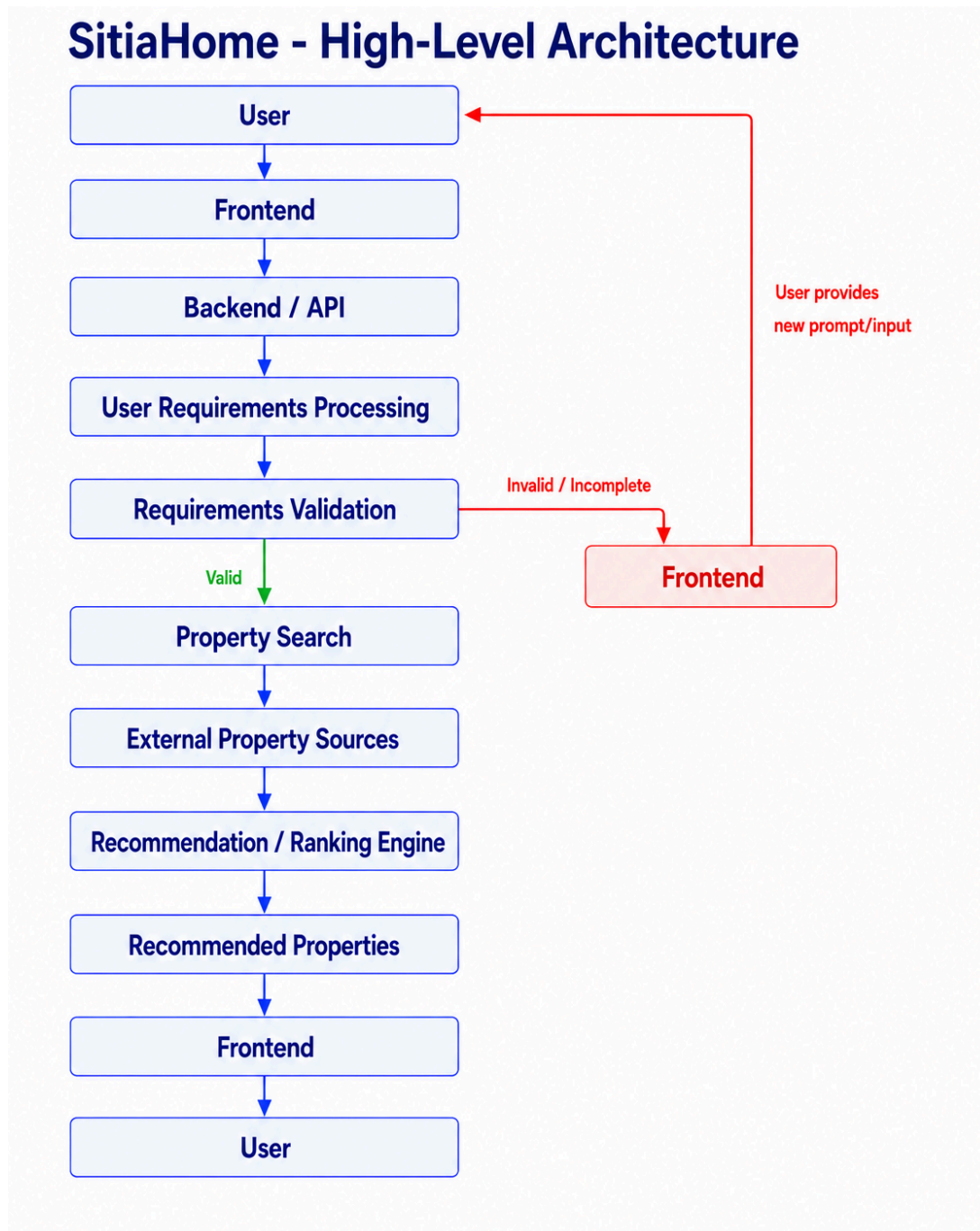
Το στάδιο του Σχεδιασμού Συστήματος αποτελεί το τρίτο στάδιο του SDLC και έχει ως στόχο να μετατρέψει τις απαιτήσεις που έχουν ήδη καθοριστεί σε ένα σαφές τεχνικό πλάνο για την υλοποίηση της εφαρμογής. Σε αυτό το στάδιο προσδιορίζεται η συνολική δομή του συστήματος, ο τρόπος με τον οποίο τα βασικά μέρη του συνεργάζονται, η ροή των δεδομένων, οι αρμοδιότητες των επιμέρους components και οι τεχνολογικές επιλογές που θα χρησιμοποιηθούν αργότερα στην υλοποίηση.

Για το SitiaHome, το στάδιο αυτό θα χωριστεί σε επιμέρους υποστάδια: Αρχιτεκτονική Υψηλού Επιπέδου (High-Level Architecture), όπου αποτυπώνεται η συνολική δομή της εφαρμογής, Ροή Δεδομένων (Data Flow), όπου περιγράφεται πώς κινούνται οι πληροφορίες μέσα στο σύστημα, Σχεδιασμός Δομικών Στοιχείων (Component Design), όπου αναλύεται ο ρόλος και η λειτουργία κάθε βασικού component και Επιλογή Τεχνολογιών (Technology Selection), όπου καθορίζονται με μεγαλύτερη λεπτομέρεια οι τεχνικές επιλογές πριν την έναρξη της υλοποίησης.

2.2.1 Αρχιτεκτονική Υψηλού Επιπέδου (High-Level Architecture)

Η Αρχιτεκτονική Υψηλού Επιπέδου αποτελεί τη συνοπτική περιγραφή της συνολικής δομής της εφαρμογής και του τρόπου με τον οποίο τα βασικά υποσυστήματα επικοινωνούν και συνεργάζονται μεταξύ τους. Σε αυτό το στάδιο δεν εξετάζονται λεπτομέρειες υλοποίησης, αλλά προσδιορίζονται τα κύρια components του συστήματος και ο ρόλος που έχει το καθένα. Στόχος της High-Level Architecture είναι να δώσει μια καθαρή συνολική εικόνα της λειτουργίας του συστήματος και να αποτελέσει τη βάση για τον αναλυτικότερο σχεδιασμό των επιμέρους components.

Το διάγραμμα της Αρχιτεκτονικής Υψηλού Επιπέδου για το σύστημα μας:



Η λογική πίσω από κάθε component είναι:

- Frontend: είναι το σημείο μέσω του οποίου ο χρήστης περιγράφει σε φυσική γλώσσα το σπίτι που αναζητά και βλέπει τις τελικές προτάσεις της εφαρμογής.
- Backend / API: αποτελεί το κεντρικό σημείο της εφαρμογής. Δέχεται το request από το Frontend και συντονίζει τη ροή και την επικοινωνία μεταξύ των επιμέρους components.
- User Requirements Processing: επεξεργάζεται την περιγραφή του χρήστη και μετατρέπει το ελεύθερο κείμενο σε οργανωμένες απαιτήσεις που μπορούν να αξιοποιηθούν από το σύστημα, όπως εύρος ενοικίου, ελάχιστος αριθμός υπνοδωματίων, περιοχή και επιπλέον προτιμήσεις.
- Requirements Validation: ελέγχει βασισμένο στο προηγούμενο βήμα εάν το prompt του χρήστη είναι έγκυρο για να πραγματοποιηθεί η αναζήτηση. Αν οι απαιτήσεις είναι έγκυρες, η διαδικασία συνεχίζεται στο Property Search. Αν είναι ελλιπείς ή μη έγκυρες, το Frontend ενημερώνει τον χρήστη και του ζητά να υποβάλει νέο ή συμπληρωμένο input.
- Property Search: είναι υπεύθυνο για την αναζήτηση διαθέσιμων κατοικιών στην πλατφόρμα ΧΕ, χρησιμοποιώντας τις επεξεργασμένες και επικυρωμένες απαιτήσεις του χρήστη.
- External Property Sources: είναι οι εξωτερικές ιστοσελίδες, APIs ή άλλες διαθέσιμες πηγές από τις οποίες η εφαρμογή αναζητά και λαμβάνει πληροφορίες για διαθέσιμες αγγελίες κατοικιών. Στην περίπτωση μας είναι η δημοφιλής πλατφόρμα για αναζήτηση κατοικίας, “Χρυσή Ευκαιρία”.
- Recommendation / Ranking Engine: αξιολογεί τις κατάλληλες αγγελίες σε σχέση με τις απαιτήσεις και τις προτιμήσεις του χρήστη. Αν οι διαθέσιμες κατάλληλες αγγελίες είναι περισσότερες από 5, επιλέγει τις 5 καταλληλότερες. Αν είναι 5 ή λιγότερες, επιστρέφονται όλες.
- Recommended Properties: αποτελούν το τελικό σύνολο κατοικιών που έχει επιλεγεί για παρουσίαση στον χρήστη, είτε όλες οι κατάλληλες αγγελίες όταν είναι έως 5 είτε οι 5 καταλληλότερες όταν ο αριθμός τους είναι μεγαλύτερος.

Η βασική ιδέα λοιπόν είναι:

Ο χρήστης περιγράφει τι κατοικία αναζητά → το σύστημα επεξεργάζεται και ελέγχει τις απαιτήσεις → αναζητά διαθέσιμες αγγελίες στην πλατφόρμα ΧΕ → αξιολογεί τα αποτελέσματα → παρουσιάζει στον χρήστη τις καταλληλότερες επιλογές

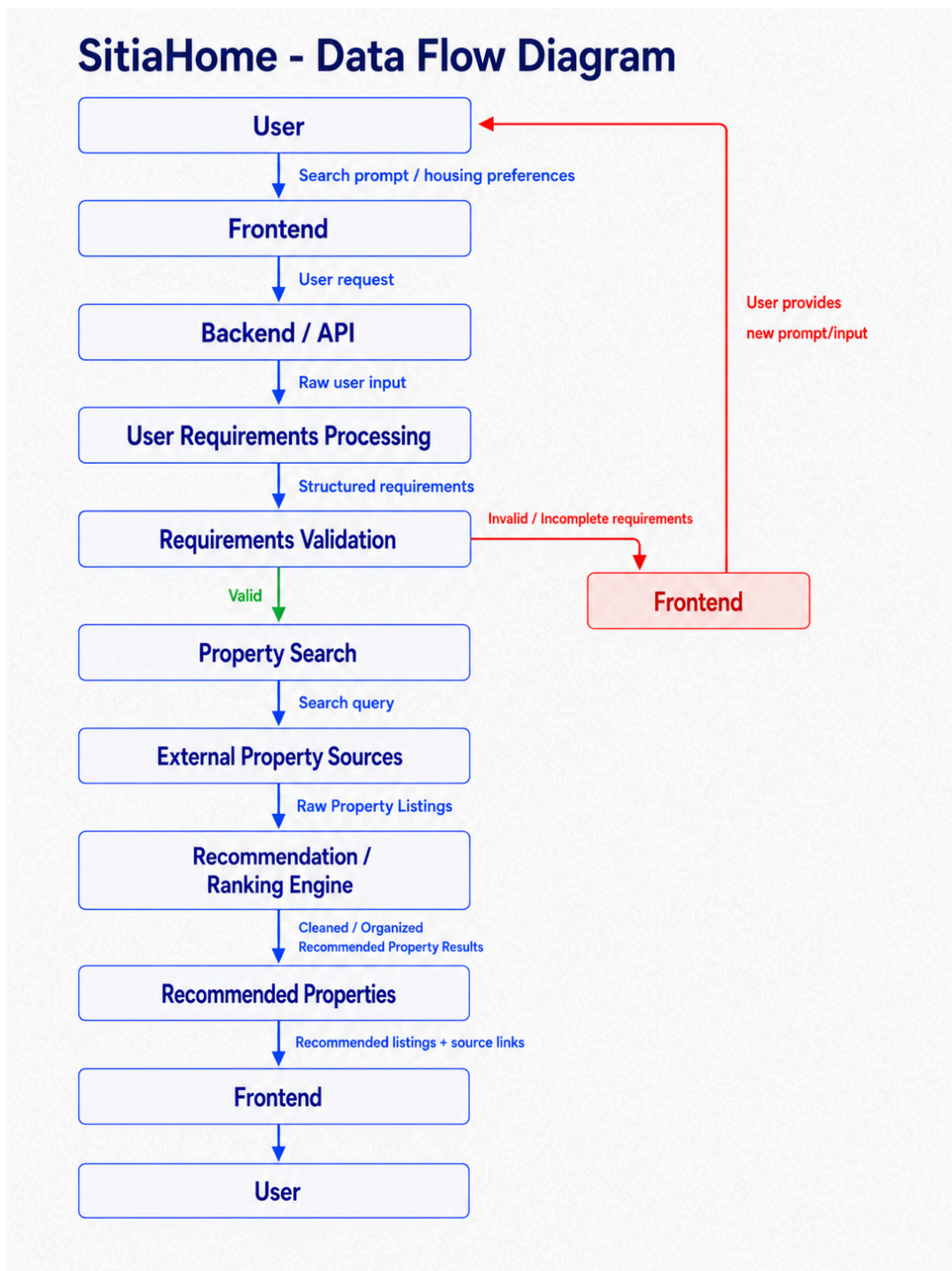
2.2.2 Ροή Δεδομένων (Data Flow)

Η **Ροή Δεδομένων** περιγράφει τον τρόπο με τον οποίο οι πληροφορίες κινούνται μέσα στην εφαρμογή από τη στιγμή που εισάγονται από τον χρήστη μέχρι να παραχθεί το τελικό αποτέλεσμα.

Στο SitiaHome, για παράδειγμα, η περιγραφή της κατοικίας που δίνει ο χρήστης μεταφέρεται από το Frontend στο Backend, γίνεται επεξεργασία της ώστε να προκύψουν οργανωμένες απαιτήσεις, ελέγχεται ως προς την εγκυρότητά της και στη συνέχεια χρησιμοποιείται για την αναζήτηση διαθέσιμων αγγελιών. Τα δεδομένα των αγγελιών που επιστρέφονται από την πλατφόρμα ΧΕ επεξεργάζονται και αξιολογούνται, ώστε να επιλεγεί το τελικό σύνολο προτεινόμενων κατοικιών.

Η αποτύπωση της ροής δεδομένων βοηθά να γίνει κατανοητό όχι μόνο ποια components συμμετέχουν στη λειτουργία του συστήματος, αλλά και ποια πληροφορία μεταφέρεται από το ένα στο άλλο και σε ποια σειρά.

Το διάγραμμα στο οποίο απεικονίζεται η ροή δεδομένων της εφαρμογής μας:



2.2.3 Σχεδιασμός Δομικών Στοιχείων (Component Design)

Ο **Σχεδιασμός Δομικών Στοιχείων** είναι το υποστάδιο του σχεδιασμού κατά το οποίο η εφαρμογή αναλύεται στα βασικά επιμέρους τμήματα που την αποτελούν και προσδιορίζεται με μεγαλύτερη σαφήνεια ο ρόλος του καθενός. Για κάθε component εξετάζεται ποια ευθύνη έχει μέσα στο σύστημα, ποια δεδομένα λαμβάνει ως είσοδο, ποια επεξεργασία πραγματοποιεί, τι αποτέλεσμα παράγει και με ποια άλλα components επικοινωνεί. Στόχος είναι να υπάρχει σαφής διαχωρισμός αρμοδιοτήτων και να γίνει κατανοητό πώς συνεργάζονται τα διαφορετικά μέρη της εφαρμογής πριν περάσουμε στις συγκεκριμένες τεχνολογίες και στην υλοποίηση.

A. Frontend

Περιγραφή

Το Frontend αποτελεί το επίπεδο της εφαρμογής μέσω του οποίου ο χρήστης αλληλεπιδρά με το SitiaHome. Είναι υπεύθυνο για την εισαγωγή της περιγραφής αναζήτησης κατοικίας, την εμφάνιση μηνυμάτων σε περίπτωση ελλειπών ή μη έγκυρων απαιτήσεων και την παρουσίαση των τελικών προτεινόμενων κατοικιών με κατανοητό και οργανωμένο τρόπο. Τέλος, στο frontend εμφανίζεται η τρέχουσα κατάσταση της εκτέλεσης της εφαρμογής για να ενημερώνεται ο χρήστης.

Είσοδος

Η βασική είσοδος του Frontend είναι η περιγραφή της κατοικίας που αναζητά ο χρήστης, η οποία δίνεται σε μορφή φυσικής γλώσσας. Το prompt μπορεί να περιλαμβάνει διαφορετικές ανάγκες και προτιμήσεις, όμως θα πρέπει υποχρεωτικά να περιέχει ορισμένα βασικά στοιχεία, όπως το μέγιστο επιθυμητό ενοίκιο και τον ελάχιστο αριθμό υπνοδωματίων. Επιπλέον, ο χρήστης μπορεί να αναφέρει προαιρετικές πληροφορίες, όπως προτιμώμενη περιοχή, επιθυμημένα χαρακτηριστικά της κατοικίας, δυνατότητα κατοικίδιων ή άλλες προσωπικές προτιμήσεις.

Βασικές Λειτουργίες

- Αποστολή του prompt στο Backend / API.
- Εμφάνιση validation messages.
- Παρουσίαση των προτεινόμενων κατοικιών.
- Εμφάνιση της τρέχουσας κατάστασης της εκτέλεσης της εφαρμογής.

Έξοδος

Το Frontend παρουσιάζει στον χρήστη τα αποτελέσματα και τα μηνύματα που προκύπτουν από την επεξεργασία του αιτήματός του. Οι βασικές έξοδοι είναι:

- Εμφάνιση μηνύματος επιτυχούς ή μη έγκυρου αιτήματος.
- Παρουσίαση των προτεινόμενων κατοικιών.
- Εμφάνιση βασικών στοιχείων για κάθε αγγελία.
- Εμφάνιση συνδέσμου προς την αρχική πηγή της αγγελίας.
- Εμφάνιση σύντομης αξιολόγησης από LLM για κάθε προτεινόμενη κατοικία με βάση τα χαρακτηριστικά της και τις απαιτήσεις του χρήστη.

Επικοινωνία με άλλα components

Το Frontend επικοινωνεί αποκλειστικά με το Backend / API, το οποίο λειτουργεί ως το ενδιάμεσο επίπεδο μεταξύ του χρήστη και των υπόλοιπων components της εφαρμογής. Μέσω αυτής της επικοινωνίας, το Frontend αποστέλλει το prompt αναζήτησης. Από το Backend λαμβάνει είτε validation messages, όταν το αίτημα είναι ελλιπές ή μη έγκυρο, είτε τις τελικές προτεινόμενες κατοικίες μαζί με τα βασικά στοιχεία τους. Η επικοινωνία με components όπως το Property Search και το Recommendation / Ranking Engine πραγματοποιείται έμμεσα, μέσω του Backend / API.

Mockup

Εύρεση κατοικίας στον Δήμο Σητείας

🔍 Περιγραφή Αναζήτησης

Ψάχνω κατοικία στη Σητεία με ενοίκιο 400€-600€, 2 δωμάτια, κοντά στο κέντρο, να επιτρέπονται κατοικίδια.

📌 Υποχρεωτικά στοιχεία: μέγιστο ενοίκιο, αριθμός δωματίων

🔍 Αναζήτηση

✅ Το αίτημα είναι έγκυρο. Η αναζήτηση μπορεί να προχωρήσει.

🏠 Προτεινόμενες Κατοικίες



Διαμέρισμα κοντά στο κέντρο

📍 Σητεία | 💰 550€ | 📄 2 δωμάτια

Φωτεινό διαμέρισμα 2ου ορόφου, κοντά στην αγορά και τις υπηρεσίες. Διαθέτει μπαλκόνι και αυτόνομη θέρμανση.

[xe.gr/property/d/12345678](#) 🔗



Studio με θέα

📍 Σητεία | 💰 420€ | 📄 1 δωμάτιο

Studio στον 2ο όροφο με όμορφη θέα στη θάλασσα, ιδανικό για ένα άτομο ή ζευγάρι. Πλήρως επιπλωμένο.

[xe.gr/property/d/87654321](#) 🔗



Κατοικία για οικογένεια

📍 Πετράς | 💰 600€ | 📄 3 δωμάτια

Άνετη μονοκατοικία με αυλή και χώρο στάθμευσης. Κατάλληλη για οικογένεια. Ήσυχη περιοχή κοντά σε σχολείο.

[xe.gr/property/d/24681357](#) 🔗

📶 Κατάσταση Εκτέλεσης

ΔΗΜΙΟΥΡΓΙΑ
ΤΕΛΙΚΩΝ ΠΡΟΤΑΣΕΩΝ



Συγκρίνουμε τις καλύτερες διαθέσιμες επιλογές με τις απαιτήσεις σας και δημιουργούμε τις τελικές προτάσεις.

16

B. Backend / API

Περιγραφή

Το Backend / API αποτελεί το κεντρικό επίπεδο συντονισμού της εφαρμογής SitiaHome και είναι υπεύθυνο για την επικοινωνία του Frontend με τα υπόλοιπα components του συστήματος. Δέχεται τα αιτήματα που προέρχονται από το Frontend, τα προωθεί στα κατάλληλα υποσυστήματα για επεξεργασία και επιστρέφει τα αποτελέσματα πίσω στον χρήστη μέσω του Frontend. Παράλληλα, διαχειρίζεται τη συνολική ροή της εφαρμογής, συντονίζοντας διαδικασίες όπως την επεξεργασία και επικύρωση των απαιτήσεων του χρήστη, την αναζήτηση διαθέσιμων κατοικιών, την αξιολόγηση και την επιλογή των καταλληλότερων αποτελεσμάτων.

Είσοδος

Η είσοδος του Backend / API προέρχεται από το Frontend και περιλαμβάνει το prompt αναζήτησης κατοικίας. Το Backend παραλαμβάνει τα δεδομένα αυτά, και τα προωθεί στο κατάλληλο component της εφαρμογής για περαιτέρω επεξεργασία.

Βασικές Λειτουργίες

- Παραλαβή και διαχείριση των αιτημάτων που αποστέλλονται από το Frontend.
- Προώθηση του prompt στο component επεξεργασίας απαιτήσεων.
- Συντονισμός του ελέγχου εγκυρότητας των απαιτήσεων του χρήστη.
- Εκκίνηση της διαδικασίας αναζήτησης κατοικιών όταν το αίτημα είναι έγκυρο.
- Συντονισμός της επεξεργασίας, αξιολόγησης και επιλογής των καταλληλότερων διαθέσιμων αγγελιών.
- Παραλαβή των τελικών προτάσεων και αποστολή τους στο Frontend.
- Επιστροφή μηνυμάτων σφάλματος ή validation feedback στο Frontend όταν απαιτείται.

Έξοδος

Η έξοδος του Backend / API αποτελείται από τα δεδομένα και τα μηνύματα που επιστρέφονται στο Frontend μετά την ολοκλήρωση των αντίστοιχων διαδικασιών. Σε αυτά περιλαμβάνονται:

- Τα μηνύματα εγκυρότητας ή σφάλματος για τα αιτήματα του χρήστη.
- Οι τελικές προτεινόμενες κατοικίες μαζί με τα βασικά στοιχεία τους.

Το Backend λειτουργεί επομένως ως το σημείο μέσω του οποίου συγκεντρώνονται τα αποτελέσματα των υπόλοιπων components και επιστρέφονται στο Frontend σε μορφή κατάλληλη για παρουσίαση στον χρήστη.

Επικοινωνία με άλλα components

Το Backend / API επικοινωνεί με όλα τα βασικά εσωτερικά components της εφαρμογής και λειτουργεί ως το κεντρικό σημείο συντονισμού τους. Λαμβάνει το αίτημα του χρήστη από το Frontend και το προωθεί στο component User Requirements Processing. Εφόσον το αίτημα είναι έγκυρο, ενεργοποιεί το Property Search για την αναζήτηση διαθέσιμων κατοικιών και στη συνέχεια προωθεί τα αποτελέσματα στο Recommendation / Ranking Engine για αξιολόγηση και επιλογή των καταλληλότερων αγγελιών. Παράλληλα, διαχειρίζεται την προσωρινή πληροφορία της τρέχουσας εκτέλεσης. Τα αποτελέσματα όλων αυτών των διαδικασιών συγκεντρώνονται από το Backend και επιστρέφονται στο Frontend για παρουσίαση στον χρήστη.

Γ. User Requirements Processing

Περιγραφή

Το User Requirements Processing (Επεξεργασία των απαιτήσεων του χρήστη) είναι το component που αναλαμβάνει να επεξεργαστεί την περιγραφή της κατοικίας που δίνει ο χρήστης σε φυσική γλώσσα και να τη μετατρέψει σε μια πιο οργανωμένη και αξιοποιήσιμη μορφή για το υπόλοιπο σύστημα. Στόχος του είναι να εντοπίζει μέσα στο prompt τις βασικές απαιτήσεις και προτιμήσεις του χρήστη, όπως το εύρος ενοικίου, τον ελάχιστο αριθμό υπνοδωματίων, την επιθυμητή περιοχή και τυχόν επιπλέον χαρακτηριστικά, ώστε αυτά να μπορούν στη συνέχεια να ελεγχθούν ως προς την εγκυρότητά τους και να χρησιμοποιηθούν από τα επόμενα components της εφαρμογής.

Είσοδος

Η είσοδος του User Requirements Processing είναι το prompt αναζήτησης που έχει εισάγει ο χρήστης στο Frontend και έχει προωθηθεί μέσω του Backend / API. Το prompt αποτελεί ελεύθερο κείμενο σε φυσική γλώσσα και μπορεί να περιλαμβάνει τόσο υποχρεωτικά στοιχεία, όπως το μέγιστο επιθυμητό ενοίκιο και τον ελάχιστο αριθμό υπνοδωματίων, όσο και προαιρετικές προτιμήσεις, όπως περιοχή, μέγεθος κατοικίας, δυνατότητα κατοικίδιων ή άλλα χαρακτηριστικά που θεωρεί σημαντικά ο χρήστης.

Βασικές Λειτουργίες

- Παραλαβή του prompt αναζήτησης από το Backend / API.
- Ανάλυση του κειμένου που έχει εισάγει ο χρήστης σε φυσική γλώσσα.
- Εντοπισμός των υποχρεωτικών απαιτήσεων, όπως το μέγιστο ενοίκιο και ο ελάχιστος αριθμός υπνοδωματίων.
- Εντοπισμός προαιρετικών προτιμήσεων, όπως περιοχή, μέγεθος κατοικίας, δυνατότητα κατοικίδιων και άλλα επιθυμητά χαρακτηριστικά.
- Μετατροπή των πληροφοριών του prompt σε οργανωμένη και δομημένη μορφή.
- Προώθηση των οργανωμένων απαιτήσεων στο component Requirements Validation για έλεγχο εγκυρότητας.

Έξοδος

Η έξοδος του User Requirements Processing είναι ένα οργανωμένο σύνολο απαιτήσεων και προτιμήσεων που έχουν εξαχθεί από το prompt του χρήστη.

Η δομημένη αυτή πληροφορία περνάει στη συνέχεια στο component Requirements Validation, ώστε να ελεγχθεί η εγκυρότητα του αιτήματος του χρήστη.

Επικοινωνία με άλλα components

Το User Requirements Processing επικοινωνεί με δύο components της εφαρμογής: το Backend / API, από το οποίο λαμβάνει το prompt του χρήστη, και το Requirements Validation, το οποίο είναι υπεύθυνο για τον έλεγχο εγκυρότητας του αιτήματος του χρήστη.

Δ. Requirements Validation

Περιγραφή

Το Requirements Validation (Έλεγχος εγκυρότητας απαιτήσεων) είναι το component που ελέγχει αν το αίτημα του χρήστη είναι έγκυρο ώστε να ξεκινήσει η διαδικασία αναζήτησης κατοικιών. Στόχος του είναι να διασφαλίσει ότι υπάρχουν όλα τα υποχρεωτικά στοιχεία, ότι οι τιμές τους είναι λογικές, και ότι το prompt δεν είναι άκυρο (ότι δηλαδή αφορά αναζήτηση κατοικίας για μακροχρόνια μίσθωση στον Δήμο Σητείας) πριν το σύστημα προχωρήσει στα επόμενα στάδια.

Να τονιστεί ότι η διαδικασία αυτή πραγματοποιείται από το ίδιο LLM που πραγματοποιεί την διαδικασία User Requirements Processing, κατά το ίδιο LLM call, απλά πρώτα ολοκληρώνεται η πρώτη διαδικασία, κι έπειτα περνάμε στην επόμενη.

Είσοδος

Η είσοδος του Requirements Validation είναι το δομημένο σύνολο απαιτήσεων που παράγεται από το User Requirements Processing. Το σύνολο αυτό περιλαμβάνει υποχρεωτικά το μέγιστο επιθυμητό ενοίκιο και τον ελάχιστο αριθμό υπνοδωματίων, ενώ μπορεί να περιλαμβάνει και προαιρετικές προτιμήσεις, όπως συγκεκριμένη περιοχή, τετραγωνικά μέτρα, δυνατότητα κατοικίδιων, επιπλωμένη κατοικία, parking ή άλλα χαρακτηριστικά που έχει αναφέρει ο χρήστης.

Βασικές Λειτουργίες

- Έλεγχος ότι υπάρχουν και τα δύο υποχρεωτικά πεδία: μέγιστο ενοίκιο και ελάχιστος αριθμός υπνοδωματίων.
- Έλεγχος ότι οι τιμές των υποχρεωτικών πεδίων είναι έγκυρες και σε σωστή μορφή.
- Έλεγχος ότι το αίτημα αφορά μακροχρόνια ενοικίαση ακινήτου στον Δήμο Σητείας.
- Χαρακτηρισμός του αιτήματος ως έγκυρο ή ελλιπές / μη έγκυρο.
- Παραγωγή κατάλληλου validation message όταν το αίτημα δεν είναι έγκυρο.
- Προώθηση των έγκυρων απαιτήσεων στο επόμενο στάδιο της αναζήτησης.

Έξοδος

Η έξοδος του Requirements Validation είναι το αποτέλεσμα του ελέγχου των απαιτήσεων του χρήστη. Αν όλα τα υποχρεωτικά πεδία υπάρχουν και οι τιμές τους είναι έγκυρες, το component επιστρέφει ένα έγκυρο σύνολο απαιτήσεων που μπορεί να χρησιμοποιηθεί από το επόμενο στάδιο της αναζήτησης. Αν κάποιο υποχρεωτικό πεδίο λείπει ή κάποια τιμή είναι μη έγκυρη, επιστρέφεται ένα validation result που περιγράφει το πρόβλημα και χρησιμοποιείται για να ζητηθεί από τον χρήστη νέο ή συμπληρωμένο input.

Επικοινωνία με άλλα components

Το Requirements Validation επικοινωνεί με το User Requirements Processing, από το οποίο λαμβάνει τις δομημένες απαιτήσεις του χρήστη, και με το Backend / API, στο οποίο επιστρέφει το αποτέλεσμα του ελέγχου. Αν οι απαιτήσεις είναι έγκυρες, το Backend συνεχίζει τη ροή προς το Property Search. Αν είναι ελλιπείς ή μη έγκυρες, το Backend επιστρέφει το κατάλληλο μήνυμα στο Frontend ώστε να ζητηθεί νέο ή συμπληρωμένο input από τον χρήστη.

E. Property search

Περιγραφή

Το Property Search είναι το component που αναλαμβάνει την αναζήτηση διαθέσιμων κατοικιών στην πλατφόρμα ΧΕ, χρησιμοποιώντας ως βάση τις έγκυρες απαιτήσεις του χρήστη. Στόχος του είναι να εντοπίζει αγγελίες που σχετίζονται με τα κριτήρια αναζήτησης, να συλλέγει τις βασικές πληροφορίες κάθε κατοικίας και να τις προωθεί στα επόμενα στάδια της εφαρμογής για περαιτέρω επεξεργασία, αξιολόγηση και επιλογή. Σε αυτό το στάδιο περιγράφεται μόνο ο λειτουργικός ρόλος του component, χωρίς ακόμη να καθορίζεται η τεχνολογία ή ο ακριβής μηχανισμός με τον οποίο θα πραγματοποιείται η αναζήτηση.

Είσοδος

Η είσοδος του Property Search είναι το έγκυρο και δομημένο σύνολο απαιτήσεων του χρήστη που έχει περάσει επιτυχώς από το Requirements Validation. Οι απαιτήσεις αυτές περιλαμβάνουν υποχρεωτικά το μέγιστο επιθυμητό ενοίκιο και τον ελάχιστο αριθμό υπνοδωματίων, ενώ μπορούν να περιλαμβάνουν και επιπλέον προτιμήσεις, όπως συγκεκριμένη περιοχή, τετραγωνικά μέτρα, δυνατότητα κατοικίδιων, επιπλωμένη κατοικία ή άλλα χαρακτηριστικά. Με βάση αυτές τις πληροφορίες, το component διαμορφώνει την αναζήτηση που θα πραγματοποιηθεί στην πλατφόρμα ΧΕ.

Βασικές Λειτουργίες

- Παραλαβή των έγκυρων και δομημένων απαιτήσεων του χρήστη.
- Αναζήτηση διαθέσιμων κατοικιών στην πλατφόρμα ΧΕ.
- Εξαγωγή των βασικών πληροφοριών από κάθε αγγελία που εντοπίζεται.
- Άμεσος έλεγχος κάθε αγγελίας ως προς τα βασικά υποχρεωτικά κριτήρια του χρήστη.
- Απόρριψη των αγγελιών που δεν ικανοποιούν τα βασικά κριτήρια ήδη κατά τη διαδικασία αναζήτησης.
- Δημιουργία λίστας με τους συνδέσμους των κατάλληλων αγγελιών.
- Προώθηση της λίστας αυτής για προσωρινή διατήρηση και στη συνέχεια για αξιολόγηση και επιλογή των καταλληλότερων αγγελιών από το Recommendation / Ranking Engine.

Έξοδος

Η έξοδος του Property Search είναι μία λίστα με τα URLs των υποψηφίων αγγελιών που έχουν ήδη περάσει τον αρχικό έλεγχο ως προς τα βασικά κριτήρια του χρήστη. Οι αγγελίες αυτές προωθούνται στη συνέχεια για προσωρινή διατήρηση και για περαιτέρω αξιολόγηση και επιλογή από το Recommendation / Ranking Engine.

Επικοινωνία με άλλα components

Το Property Search επικοινωνεί με το Backend / API, από το οποίο λαμβάνει τις έγκυρες απαιτήσεις αναζήτησης, και με τις External Property Sources, από τις οποίες αντλεί τις διαθέσιμες αγγελίες. Αφού εντοπίσει και φιλτράρει τις κατάλληλες αγγελίες, προωθεί τους συνδέσμους τους στο Recommendation / Ranking Engine, ώστε να γίνει η επιλογή των καταλληλότερων αγγελιών.

Z. Recommendation / Ranking Engine

Περιγραφή

Το Recommendation / Ranking Engine είναι το component που αναλαμβάνει την περαιτέρω επεξεργασία και αξιολόγηση των υποψήφιων αγγελιών που έχουν προκύψει από το Property Search. Για κάθε υποψήφια κατοικία ανακτώνται αναλυτικότερες πληροφορίες από την αντίστοιχη αγγελία της Χρυσής Ευκαιρίας και οργανώνονται σε κοινή structured μορφή. Στη συνέχεια, κάθε κατοικία αξιολογείται σε σχέση τόσο με τα χαρακτηριστικά της όσο και με τις απαιτήσεις και προτιμήσεις του χρήστη.

Για κάθε ακίνητο παράγεται επίσης μία σύντομη αξιολόγηση σε φυσική γλώσσα, η οποία παρουσιάζει τα σημαντικότερα θετικά και αρνητικά στοιχεία και πιθανά trade-offs σε σχέση με τις ανάγκες του χρήστη. Αν οι διαθέσιμες υποψήφιες κατοικίες είναι έως 5, επιστρέφονται όλες. Αν είναι περισσότερες από 5, πραγματοποιείται τελικό στάδιο επιλογής και κατάταξης, ώστε να επιστραφούν οι 5 καταλληλότερες σε σειρά από την καλύτερη προς την ασθενέστερη αντιστοίχιση.

Είσοδος

Η είσοδος του Recommendation / Ranking Engine αποτελείται από δύο βασικά σύνολα δεδομένων:

- τις δομημένες απαιτήσεις και προτιμήσεις του χρήστη,
- τη λίστα με τα URLs των υποψήφιων αγγελιών που έχουν προκύψει από το Property Search.

Μέσω των URLs, το component αποκτά πρόσβαση στα αναλυτικότερα δεδομένα κάθε αγγελίας και τα μετατρέπει σε κοινή structured μορφή, η οποία περιλαμβάνει στοιχεία όπως τίτλο, τιμή, περιοχή, τετραγωνικά μέτρα, αριθμό υπνοδωματίων, κατάσταση επίπλωσης, περιγραφή και σύνδεσμο προς την αρχική αγγελία.

Βασικές Λειτουργίες

- Παραλαβή των URLs των υποψήφιων αγγελιών.
- Παραλαβή των δομημένων απαιτήσεων και προτιμήσεων του χρήστη.
- Ανάκτηση των αναλυτικών δεδομένων κάθε αγγελίας από την πλατφόρμα ΧΕ.
- Εξαγωγή και οργάνωση των στοιχείων κάθε κατοικίας σε κοινό structured schema.
- Παραγωγή αξιολόγησης για κάθε κατοικία με βάση τα χαρακτηριστικά της και τις απαιτήσεις του χρήστη.
- Συνδυασμός των structured στοιχείων και της αξιολόγησης κάθε κατοικίας σε ενιαίο αποτέλεσμα.
- Αν οι διαθέσιμες κατοικίες είναι έως 5, επιστροφή όλων.
- Αν είναι περισσότερες από 5, επιλογή και κατάταξη των 5 καταλληλότερων.
- Προώθηση των τελικών προτεινόμενων κατοικιών στο Backend / API.

Έξοδος

Η έξοδος του Recommendation / Ranking Engine είναι το τελικό σύνολο προτεινόμενων κατοικιών που θα παρουσιαστεί στον χρήστη. Κάθε αποτέλεσμα περιλαμβάνει τα οργανωμένα στοιχεία της κατοικίας, μία σύντομη αξιολόγησή της σε σχέση με τις απαιτήσεις του χρήστη και τον σύνδεσμο προς την αρχική αγγελία στη Χρυσή Ευκαιρία.

Αν οι διαθέσιμες κατοικίες είναι 5 ή λιγότερες, επιστρέφονται όλες. Αν είναι περισσότερες από 5, επιστρέφονται οι 5 καταλληλότερες σε ταξινομημένη σειρά.

Επικοινωνία με άλλα Components

Το Recommendation / Ranking Engine επικοινωνεί με το Property Search, από το οποίο λαμβάνει τα URLs των υποψήφιων αγγελιών, και με το Backend / API, από το οποίο λαμβάνει τις δομημένες απαιτήσεις του χρήστη και στο οποίο επιστρέφει τις τελικές προτεινόμενες κατοικίες. Παράλληλα,

προσπελαύνει τις σελίδες των επιμέρους αγγελιών στην πλατφόρμα ΧΕ για την ανάκτηση των αναλυτικών χαρακτηριστικών τους.

2.2.4 Επιλογή Τεχνολογιών

Στην υποενότητα Επιλογή Τεχνολογιών θα καθοριστούν οι συγκεκριμένες τεχνολογίες, τα frameworks και τα εργαλεία που θα χρησιμοποιηθούν για την υλοποίηση των επιμέρους components του SitiaHome. Η επιλογή θα πραγματοποιηθεί ξεχωριστά για κάθε component, λαμβάνοντας υπόψη τον ρόλο του στο σύστημα, τις λειτουργικές και μη λειτουργικές απαιτήσεις που έχουν ήδη οριστεί, την ευκολία ενσωμάτωσης με τα υπόλοιπα μέρη της εφαρμογής, καθώς και παράγοντες όπως η επεκτασιμότητα, η συντηρησιμότητα και η καταλληλότητα για ένα εκπαιδευτικό Software/AI Engineering project.

A. Frontend

Για την υλοποίηση του **Frontend** επιλέχθηκε το **React** [2], καθώς προσφέρει έναν οργανωμένο και modular τρόπο ανάπτυξης διαδραστικών user interfaces μέσω επαναχρησιμοποιήσιμων components. Ταιριάζει στις ανάγκες του SitiaHome, όπου το περιβάλλον χρήστη περιλαμβάνει δυναμικά στοιχεία όπως η εισαγωγή του prompt, τα validation messages, η εμφάνιση της κατάστασης εκτέλεσης και η παρουσίαση των προτεινόμενων αγγελιών. Επιπλέον, το React διευκολύνει τη διαχείριση της κατάστασης του UI και την επικοινωνία με το Backend μέσω API, διατηρώντας παράλληλα τον κώδικα οργανωμένο και επεκτάσιμο. Για τη δημιουργία και εκτέλεση του React frontend χρησιμοποιείται το **Vite** [7], ένα σύγχρονο development και build tool που παρέχει γρήγορη εκκίνηση του development server και άμεση ανανέωση της εφαρμογής κατά την ανάπτυξη.

B. Backend

Για την υλοποίηση του **Backend / API** επιλέχθηκε το **FastAPI** [3], ένα σύγχρονο Python framework για την ανάπτυξη web APIs. Η επιλογή του ταιριάζει στη δομή του SitiaHome, καθώς το Backend λειτουργεί ως το

κεντρικό επίπεδο συντονισμού μεταξύ του Frontend και των υπόλοιπων components της εφαρμογής. Το FastAPI επιτρέπει την εύκολη δημιουργία API endpoints και την οργανωμένη διαχείριση requests και responses, ενώ η χρήση της Python διευκολύνει την ενσωμάτωση των AI, data processing και recommendation λειτουργιών που χρησιμοποιούνται στα επόμενα components του συστήματος. Για την εκτέλεση της FastAPI εφαρμογής χρησιμοποιείται ο **Uvicorn** [6], ένας ASGI server που αναλαμβάνει να εκκινεί το Backend και να δέχεται τα HTTP requests που αποστέλλονται από το Frontend προς τα διαθέσιμα API endpoints.

Γ. User Requirements Processing

Για την υλοποίηση του **User Requirements Processing** θα χρησιμοποιηθεί το Large Language Model (LLM) GPT-5.6 Luna με δυνατότητα Structured Output. Το μοντέλο θα λαμβάνει την περιγραφή του χρήστη σε φυσική γλώσσα και θα τη μετατρέπει σε μια προκαθορισμένη δομή δεδομένων. Το structured output περιλαμβάνει το πεδίο legit_prompt, το προαιρετικό ελάχιστο επιθυμητό ενοίκιο (min_rent), το υποχρεωτικό μέγιστο επιθυμητό ενοίκιο (max_rent), τον υποχρεωτικό ελάχιστο αριθμό υπνοδωματίων (min_bedrooms), ένα πεδίο επιπλέον πληροφοριών (additional_information) και το validation_message. Στο additional_information θα συγκεντρώνονται όλες οι υπόλοιπες απαιτήσεις ή προτιμήσεις του χρήστη, όπως περιοχή, τετραγωνικά μέτρα, επιπλωμένη κατοικία, δυνατότητα κατοικίδιων, parking ή άλλα χαρακτηριστικά. Με αυτόν τον τρόπο, το ελεύθερο κείμενο του χρήστη μετατρέπεται σε μια σταθερή και οργανωμένη μορφή που μπορεί να αξιοποιηθεί από το Requirements Validation και τα επόμενα components της εφαρμογής.

Δ. Requirements Validation

Το **Requirements Validation** αποτελεί άμεση συνέχεια του User Requirements Processing και βασίζεται στο structured output που έχει ήδη παραχθεί από το LLM. Στο στάδιο αυτό ελέγχονται από το ίδιο μοντέλο με προηγουμένως τα πεδία της δομημένης εξόδου που προέκυψε από το προηγούμενο βήμα, με σκοπό να δοθεί τιμή στο πεδίο legit_prompt. Υπάρχει και η μεταβλητή validation_message που παίρνει τιμή “OK” αν το αίτημα είναι έγκυρο, ή στην περίπτωση που δεν είναι λαμβάνει την αιτία που συμβαίνει αυτό. Οι τιμές όλων των πεδίων αυτών συμπληρώνονται από το LLM. Το

legit_prompt λαμβάνει την τιμή false στις εξής περιπτώσεις: α) Το prompt του χρήστη δεν αφορά εύρεση κατοικίας για μακροχρόνια μίσθωση στον Δήμο Σητείας, β) Κάποιο εκ των υποχρεωτικών πεδίων είναι κενό, γ) Κάποια τιμή είναι παράλογη, π.χ. αρνητικό μέγιστο ενοίκιο ή ελάχιστος αριθμός υπνοδωματίων πολύ μεγάλος.

E. Property Search

Για την υλοποίηση του **Property Search** χρησιμοποιείται ως εξωτερική πηγή αγγελιών η πλατφόρμα Χρυσή Ευκαιρία (ΧΕ) [4]. Το component λαμβάνει τις δομημένες και επικυρωμένες απαιτήσεις του χρήστη και αναλαμβάνει να εντοπίσει τις αγγελίες που ικανοποιούν τα βασικά υποχρεωτικά κριτήρια αναζήτησης.

Η ανάκτηση των διαθέσιμων αγγελιών πραγματοποιείται προγραμματιστικά μέσω HTTP requests προς τη σελίδα αποτελεσμάτων της ΧΕ. Τα δεδομένα των αγγελιών είναι διαθέσιμα σε structured μορφή μέσα στο HTML της σελίδας, εξάγονται με χρήση της βιβλιοθήκης bs4 [5], και μετατρέπονται σε κατάλληλη Python δομή για περαιτέρω επεξεργασία.

Στη συνέχεια πραγματοποιείται αρχικό filtering με βάση το μέγιστο επιθυμητό ενοίκιο και τον ελάχιστο αριθμό υπνοδωματίων, καθώς και με βάση το ελάχιστο ενοίκιο όταν αυτό έχει δηλωθεί από τον χρήστη. Οι αγγελίες που δεν ικανοποιούν τα συγκεκριμένα κριτήρια απορρίπτονται, ενώ για όσες παραμένουν διατηρούνται οι σύνδεσμοι προς τις αντίστοιχες σελίδες τους. Στο επόμενο στάδιο θα ανακτηθούν αναλυτικότερες πληροφορίες από τις επιμέρους αγγελίες και θα οργανωθούν σε κοινό structured schema, ώστε να χρησιμοποιηθούν από το Recommendation / Ranking component. Η συγκεκριμένη προσέγγιση διαχωρίζει τη διαδικασία αναζήτησης και αρχικού filtering από την πιο σύνθετη αξιολόγηση των αποτελεσμάτων.

Z. Recommendation / Ranking Engine

Για την υλοποίηση του Recommendation / Ranking Engine χρησιμοποιείται το LLM GPT-5.4 mini σε συνδυασμό με structured outputs. Για κάθε υποψήφια αγγελία πραγματοποιείται HTTP request προς την αντίστοιχη σελίδα της Χρυσής Ευκαιρίας και εξάγεται το structured property data που βρίσκεται στο HTML με χρήση της bs4. Το περιεχόμενο αυτό επεξεργάζεται από το LLM και μετατρέπεται σε κοινό structured schema PropertyListing, το οποίο περιλαμβάνει στοιχεία όπως τίτλο, τιμή, περιοχή, τετραγωνικά μέτρα, αριθμό υπνοδωματίων, κατάσταση επίπλωσης, περιγραφή και URL.

Για κάθε κατοικία παράγεται επίσης μία σύντομη αξιολόγηση σε φυσική γλώσσα από το LLM, λαμβάνοντας υπόψη τόσο τα χαρακτηριστικά της αγγελίας όσο και τις απαιτήσεις και προτιμήσεις του χρήστη. Η αξιολόγηση αποθηκεύεται μαζί με τα στοιχεία της κατοικίας σε ένα FinalProperty.

Αν οι διαθέσιμες κατοικίες είναι έως πέντε, επιστρέφονται όλες. Αν είναι περισσότερες από πέντε, πραγματοποιείται δεύτερο LLM-based στάδιο επιλογής και κατάταξης, στο οποίο επιλέγονται οι πέντε καταλληλότερες και επιστρέφονται σε σειρά από την καλύτερη προς την ασθενέστερη αντιστοίχιση.

2.3 Υλοποίηση

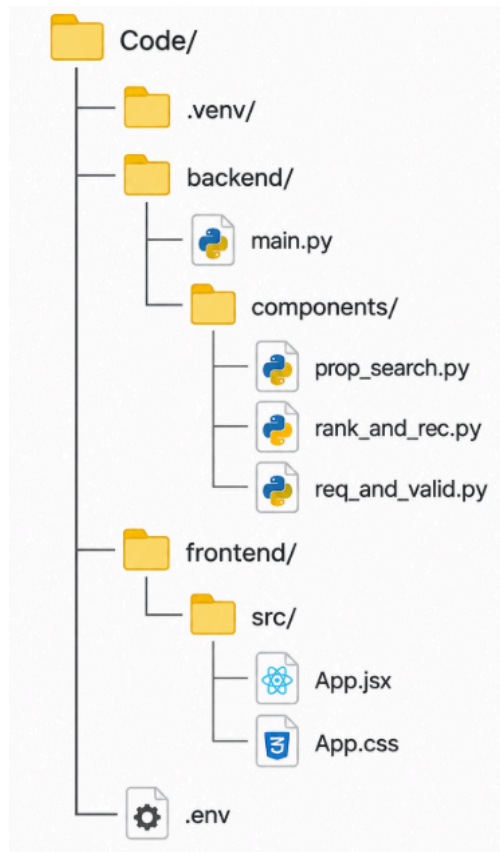
Στο στάδιο της **Υλοποίησης** οι απαιτήσεις και οι σχεδιαστικές αποφάσεις που καθορίστηκαν στα προηγούμενα στάδια του project μετατρέπονται σε λειτουργικό λογισμικό. Η ενότητα αυτή παρουσιάζει τον τρόπο με τον οποίο οργανώνεται και υλοποιείται στην πράξη το SitiaHome, ξεκινώντας από τη συνολική δομή φακέλων και αρχείων (folder/file structure) του project. Η παρουσίαση της δομής αυτής βοηθά στην κατανόηση του διαχωρισμού των διαφορετικών μερών της εφαρμογής, όπως το Frontend, το Backend και τα επιμέρους modules επεξεργασίας απαιτήσεων, αναζήτησης ακινήτων και recommendation/ranking.

Στη συνέχεια παρουσιάζεται ξεχωριστά η υλοποίηση κάθε βασικού component του συστήματος. Για κάθε module παρατίθενται επιλεγμένα αποσπάσματα κώδικα και εξηγούνται ο ρόλος του, οι βασικές functions που περιλαμβάνει, τα δεδομένα που δέχεται ως είσοδο, η επεξεργασία που πραγματοποιεί και το αποτέλεσμα που παράγει. Με αυτόν τον τρόπο γίνεται σαφής η ευθύνη κάθε τμήματος της εφαρμογής και ο τρόπος με τον οποίο αυτό εντάσσεται στη συνολική αρχιτεκτονική του SitiaHome.

Παράλληλα, τεκμηριώνονται οι σημαντικότερες τεχνικές επιλογές που προκύπτουν κατά την ανάπτυξη, όπως η δημιουργία των API endpoints, ο ορισμός των structured data models, η ενσωμάτωση των LLMs, η προσωρινή διαχείριση των δεδομένων και η υλοποίηση των μηχανισμών recommendation και ranking. Στόχος δεν είναι να παρατεθεί ολόκληρος ο πηγαίος κώδικας του project, αλλά να παρουσιαστούν τα τμήματα που είναι σημαντικά για την κατανόηση της λειτουργίας και των βασικών τεχνικών μηχανισμών του συστήματος.

Στο τέλος της ενότητας θα παρουσιαστούν διαφορετικά παραδείγματα εκτέλεσης της εφαρμογής, ώστε να εξεταστεί η συμπεριφορά της σε διαφορετικά σενάρια χρήσης και διαφορετικές πιθανές εκβάσεις της συνολικής ροής. Ενδεικτικά, θα παρουσιαστούν περιπτώσεις μη έγκυρου ή ελλιπούς αιτήματος, περιπτώσεις στις οποίες δεν εντοπίζονται κατάλληλες κατοικίες, καθώς και περιπτώσεις επιτυχούς αναζήτησης όπου παράγονται και εμφανίζονται οι τελικές προτεινόμενες κατοικίες. Με αυτόν τον τρόπο θα αποτυπωθεί στην πράξη τόσο η κανονική λειτουργία της εφαρμογής όσο και ο τρόπος με τον οποίο το σύστημα διαχειρίζεται διαφορετικές καταστάσεις και αποτελέσματα κατά την εκτέλεσή του.

Το ακόλουθο folder/file structure περιλαμβάνει μόνο τα αρχεία και τους φακέλους που είναι απαραίτητοι για την παρουσίαση της υλοποίησης και αναλύονται στη συνέχεια.



2.3.1 Εκτέλεση και Αρχικοποίηση της Εφαρμογής

Στην ενότητα αυτή θα παρουσιαστεί ο τρόπος με τον οποίο προετοιμάζεται και εκκινείται το SitiaHome στο τοπικό περιβάλλον ανάπτυξης. Συγκεκριμένα, θα περιγραφεί η ενεργοποίηση του Python virtual environment, η φόρτωση των απαραίτητων μεταβλητών περιβάλλοντος από το .env, η εκκίνηση του Backend μέσω FastAPI/Uvicorn και η εκκίνηση του Frontend μέσω του React development server. Παράλληλα, θα αναφερθεί ο τρόπος με τον οποίο τα δύο μέρη της εφαρμογής επικοινωνούν μεταξύ τους μέσω των αντίστοιχων τοπικών διευθύνσεων και API endpoints, ώστε να είναι διαθέσιμη η πλήρης λειτουργία του συστήματος πριν εξεταστεί αναλυτικά η εσωτερική ροή εκτέλεσης.

.venv

Το .venv είναι το Python virtual environment του project. Χρησιμοποιείται ώστε οι Python βιβλιοθήκες και οι εξαρτήσεις του SitiaHome να εγκαθίστανται σε ένα απομονωμένο περιβάλλον και να μην επηρεάζουν ή επηρεάζονται από άλλες Python εγκαταστάσεις και projects του υπολογιστή. Με αυτόν τον τρόπο,

το Backend μπορεί να χρησιμοποιεί συγκεκριμένες εκδόσεις βιβλιοθηκών όπως FastAPI, httpx, BeautifulSoup, Pydantic και OpenAI, διατηρώντας το περιβάλλον του project οργανωμένο και αναπαραγώγιμο. Πριν την εκκίνηση του Backend, το virtual environment ενεργοποιείται ώστε οι εντολές Python και οι εγκατεστημένες dependencies να εκτελούνται μέσα από αυτό.

Η ενεργοποίηση του virtual environment πραγματοποιείται από το terminal, μέσα από τον φάκελο του project. Στα Windows μπορεί να γίνει, για παράδειγμα, με την εντολή:

```
.venv\Scripts\activate
```

Μετά την ενεργοποίηση, το terminal χρησιμοποιεί τον Python interpreter και τα packages του συγκεκριμένου virtual environment αντί για τις global εγκαταστάσεις του συστήματος. Οι βιβλιοθήκες που εγκαθίστανται με pip, αποθηκεύονται τοπικά μέσα στον φάκελο .venv, συνήθως σε υποφακέλους όπως Lib/site-packages. Με αυτόν τον τρόπο, οι dependencies του SitiaHome παραμένουν απομονωμένες από άλλα projects και από την κεντρική εγκατάσταση της Python.

.env

Το αρχείο .env χρησιμοποιείται για την αποθήκευση μεταβλητών περιβάλλοντος και ευαίσθητων ρυθμίσεων που δεν είναι σωστό να γράφονται απευθείας μέσα στον πηγαίο κώδικα, όπως για παράδειγμα ένα API key.

Στο SitiaHome το .env βρίσκεται στον βασικό φάκελο του project και περιλαμβάνει την απαραίτητη τιμή για την πρόσβαση στο OpenAI API. Η φόρτωσή του στον Python κώδικα γίνεται με τη βιβλιοθήκη python-dotenv, μέσω της εντολής load_dotenv(). Μετά την εκτέλεση της συγκεκριμένης εντολής, οι μεταβλητές του .env γίνονται διαθέσιμες ως environment variables και μπορούν να χρησιμοποιηθούν από τις βιβλιοθήκες και τα components της εφαρμογής χωρίς να χρειάζεται να εμφανίζονται μέσα στον κώδικα. Για λόγους ασφάλειας, το .env δεν πρέπει να ανεβαίνει σε δημόσιο repository, όπως το GitHub.

Υλοποίηση και Εκτέλεση του Backend με FastAPI και Unicorn

Το Backend του SitiaHome υλοποιείται με χρήση του **FastAPI**, το οποίο αποτελεί το βασικό framework για τον ορισμό της εφαρμογής και των διαθέσιμων API endpoints. Στο αρχείο `main.py` δημιουργείται το αντικείμενο `app = FastAPI()` και ορίζονται τα endpoints μέσω των οποίων το Frontend επικοινωνεί με το Backend, όπως τα `/search`, `/search-properties` και `/select-properties`. Μέσω αυτών των endpoints, το Backend παραλαμβάνει τα requests του Frontend, καλεί τα κατάλληλα components της εφαρμογής και επιστρέφει τα αντίστοιχα αποτελέσματα. Παράλληλα, μέσω του FastAPI middleware ρυθμίζεται και το CORS, ώστε να επιτρέπεται η επικοινωνία με το Frontend που εκτελείται σε διαφορετική τοπική διεύθυνση και port.

Για την εκτέλεση της FastAPI εφαρμογής χρησιμοποιείται ο **Uvicorn**, ο οποίος λειτουργεί ως ASGI server. Αρχικά ενεργοποιείται το virtual environment του project, ώστε να χρησιμοποιούνται οι σωστές Python dependencies, και στη συνέχεια το Backend εκκινείται από το terminal με την εντολή:

```
uvicorn backend.main:app --reload
```

Η εντολή αυτή υποδεικνύει στον Uvicorn να εντοπίσει το αντικείμενο `app` μέσα στο `backend/main.py` και να το εκτελέσει ως ASGI application. Η επιλογή `--reload` χρησιμοποιείται κατά τη διαδικασία development, ώστε ο server να επανεκκινείται αυτόματα όταν πραγματοποιούνται αλλαγές στον Python κώδικα. Μετά την εκκίνηση, το Backend είναι διαθέσιμο τοπικά στη διεύθυνση `http://127.0.0.1:8000`, από όπου δέχεται και διαχειρίζεται τα HTTP requests που αποστέλλονται από το Frontend.

Υλοποίηση και Εκτέλεση του Frontend με React και Vite

Το Frontend του SitiaHome υλοποιείται με **React**, με τον βασικό κώδικα της διεπαφής να βρίσκεται στο αρχείο `App.jsx`, ενώ η εμφάνιση και η μορφοποίηση των στοιχείων της εφαρμογής ορίζονται στο `App.css`. Μέσω του React διαχειρίζονται τα διαφορετικά states της εφαρμογής, όπως το prompt του χρήστη, τα validation messages, η κατάσταση εκτέλεσης και οι τελικές προτεινόμενες κατοικίες. Παράλληλα, από το `App.jsx` πραγματοποιούνται τα HTTP requests προς το Backend, ώστε το Frontend να αποστέλλει τα δεδομένα του χρήστη και να λαμβάνει τα αντίστοιχα αποτελέσματα.

Για την τοπική εκτέλεση του Frontend χρησιμοποιείται το **Vite**, το οποίο λειτουργεί ως development server και build tool για την React εφαρμογή. Η εκκίνηση πραγματοποιείται από το terminal μέσα από τον φάκελο του

Frontend με την εντολή `npm run dev`. Με την εντολή αυτή ξεκινά ο Vite development server και η εφαρμογή γίνεται διαθέσιμη τοπικά, συνήθως στη διεύθυνση `http://localhost:5173`. Κατά τη διάρκεια της ανάπτυξης, το Vite επιτρέπει την άμεση ανανέωση της εφαρμογής όταν γίνονται αλλαγές στον frontend κώδικα, διευκολύνοντας τη διαδικασία development και testing.

Επικοινωνία μεταξύ Frontend και Backend

Η επικοινωνία μεταξύ του Frontend και του Backend στο SitiaHome πραγματοποιείται μέσω HTTP requests προς τα API endpoints που έχουν οριστεί στο FastAPI. Το Frontend, το οποίο εκτελείται με React και Vite, χρησιμοποιεί τη συνάρτηση `fetch()` για να αποστέλλει αιτήματα προς το Backend. Για παράδειγμα, όταν ο χρήστης εισάγει το prompt αναζήτησης και πατήσει το κουμπί αναζήτησης, το Frontend στέλνει το περιεχόμενο του prompt στο endpoint `/search`. Το Backend επεξεργάζεται το αίτημα μέσω του αντίστοιχου component και επιστρέφει την απάντηση σε μορφή JSON, την οποία το React χρησιμοποιεί για να ενημερώσει το state και την εμφάνιση της εφαρμογής.

Η ίδια λογική ακολουθείται και στα επόμενα στάδια της ροής, με το Frontend να καλεί διαδοχικά τα endpoints `/search-properties` και `/select-properties`, ανάλογα με το αποτέλεσμα των προηγούμενων βημάτων. Με αυτόν τον τρόπο, το Frontend δεν επικοινωνεί απευθείας με τα επιμέρους backend components, αλλά μόνο με το FastAPI API, το οποίο λειτουργεί ως ενδιάμεσο επίπεδο συντονισμού. Επειδή το Frontend εκτελείται τοπικά στο `http://localhost:5173` και το Backend στο `http://127.0.0.1:8000`, έχει ενεργοποιηθεί CORS middleware στο FastAPI ώστε να επιτρέπονται requests μεταξύ των δύο διαφορετικών origins. Έτσι, το Frontend και το Backend παραμένουν ξεχωριστά επίπεδα της εφαρμογής, αλλά συνεργάζονται μέσω ενός σαφώς ορισμένου API-based μηχανισμού επικοινωνίας.

2.3.2 Ανάλυση Κώδικα και Ροής Εκτέλεσης

Στην ενότητα αυτή θα παρουσιαστεί η υλοποίηση του SitiaHome μέσα από τα βασικά αρχεία και modules που αποτελούν την εφαρμογή. Η ανάλυση θα πραγματοποιηθεί ξεχωριστά για κάθε σημαντικό τμήμα του συστήματος, με παρουσίαση του ρόλου του, των βασικών functions, των δεδομένων που διαχειρίζεται και των σημαντικότερων αποσπασμάτων κώδικα που είναι απαραίτητα για την κατανόηση της λειτουργίας του.

Με αυτόν τον τρόπο θα εξεταστούν διαδοχικά το Frontend, το Backend και τα επιμέρους components επεξεργασίας απαιτήσεων, αναζήτησης ακινήτων και recommendation/ranking. Στόχος είναι να αποτυπωθεί με σαφήνεια η υλοποίηση και η ευθύνη κάθε module χωρίς να παρατεθεί ολόκληρος ο πηγαίος κώδικας.

App.jsx

Το αρχείο **App.jsx** αποτελεί το βασικό React component του Frontend του SitiaHome και συγκεντρώνει την κύρια λογική της διεπαφής της εφαρμογής. Ο ρόλος του δεν περιορίζεται μόνο στην εμφάνιση των οπτικών στοιχείων, αλλά περιλαμβάνει και τη διαχείριση της κατάστασης του Frontend, την επεξεργασία των ενεργειών του χρήστη, την επικοινωνία με το Backend και την παρουσίαση των αποτελεσμάτων που επιστρέφονται από αυτό. Η εφαρμογή έχει υλοποιηθεί ως functional React component, δηλαδή ως JavaScript function που επιστρέφει JSX και ενημερώνεται δυναμικά όταν αλλάζουν τα δεδομένα της εφαρμογής. Στην αρχή του αρχείου εισάγεται το useState από τη React, καθώς και το App.css, το οποίο περιλαμβάνει το styling της διεπαφής.

Το useState είναι ένα React Hook που επιτρέπει στο component να διατηρεί και να μεταβάλλει δεδομένα κατά τη διάρκεια της εκτέλεσης της εφαρμογής. Κάθε κλήση του useState επιστρέφει δύο στοιχεία: την τρέχουσα τιμή ενός state variable και μία function μέσω της οποίας η τιμή αυτή μπορεί να ενημερωθεί. Για παράδειγμα, στο `const [prompt, setPrompt] = useState("")`, το prompt περιέχει το κείμενο που έχει εισάγει ο χρήστης, ενώ η setPrompt() χρησιμοποιείται για να μεταβάλλει αυτή την τιμή. Με τον ίδιο τρόπο ορίζονται όλα τα state variables που χρειάζεται το Frontend για να γνωρίζει σε ποιο σημείο της διαδικασίας βρίσκεται και τι πρέπει να εμφανίζει στον χρήστη.

Το prompt συνδέεται με το πεδίο κειμένου στο οποίο ο χρήστης περιγράφει την κατοικία που αναζητά. Το message χρησιμοποιείται για την εμφάνιση

validation ή error messages, ενώ το `isValid` επιτρέπει στο Frontend να γνωρίζει αν το συγκεκριμένο μήνυμα αφορά επιτυχή ή αποτυχημένο έλεγχο. Το `loading` δηλώνει αν βρίσκεται σε εξέλιξη η συνολική διαδικασία αναζήτησης και χρησιμοποιείται, μεταξύ άλλων, για να απενεργοποιείται προσωρινά το κουμπί αναζήτησης και να αποτρέπεται η αποστολή πολλαπλών αιτημάτων. Τα `executionTitle` και `executionDescription` χρησιμοποιούνται αποκλειστικά για το panel κατάστασης εκτέλεσης και ενημερώνονται ανάλογα με το στάδιο στο οποίο βρίσκεται η εφαρμογή. Τέλος, το `recommendations` είναι ένας πίνακας στον οποίο αποθηκεύονται τα τελικά αποτελέσματα που επιστρέφει το Backend. Όταν κάποιο από αυτά τα `state variables` αλλάξει, το React ενημερώνει αυτόματα τα αντίστοιχα στοιχεία της διεπαφής.

Η βασική λειτουργική λογική του Frontend υλοποιείται μέσα στη function **`handleSearch()`**, η οποία καλείται όταν ο χρήστης πατήσει το κουμπί «Αναζήτηση». Το πρώτο βήμα είναι ένας απλός client-side έλεγχος για το αν το `prompt` είναι κενό. Η χρήση της `trim()` αφαιρεί κενά από την αρχή και το τέλος του string, ώστε ένα `input` που αποτελείται μόνο από κενά να θεωρείται επίσης κενό. Σε αυτή την περίπτωση η διαδικασία σταματά πριν γίνει οποιοδήποτε HTTP request προς το Backend, εμφανίζεται σχετικό μήνυμα στον χρήστη και καθαρίζονται τυχόν προηγούμενα αποτελέσματα ή πληροφορίες εκτέλεσης.

```
if (!prompt.trim()) {  
  setIsValid(false);  
  setMessage("Παρακαλώ περιγράψτε την κατοικία που αναζητάτε.");  
  setExecutionTitle("");  
  setExecutionDescription("");  
  setRecommendations([]);  
  return;  
}
```

Αν το `prompt` περιέχει κείμενο, το Frontend προετοιμάζει τη νέα εκτέλεση. Το `loading` τίθεται σε `true`, ώστε να θεωρείται ότι η διαδικασία βρίσκεται σε εξέλιξη, ενώ καθαρίζονται τυχόν προηγούμενα `messages` και `recommendations`. Παράλληλα, ενημερώνεται η περιοχή «Κατάσταση Εκτέλεσης» με τίτλο και περιγραφή που δείχνουν ότι το αίτημα ελέγχεται ως προς την εγκυρότητά του. Αυτό είναι σημαντικό από άποψη `user experience`, επειδή ο χρήστης γνωρίζει ότι η εφαρμογή δεν έχει «παγώσει», αλλά εκτελεί συγκεκριμένο στάδιο της διαδικασίας.

Στη συνέχεια πραγματοποιείται το πρώτο HTTP request προς το Backend. Χρησιμοποιείται η JavaScript `fetch()` για την αποστολή POST request στο endpoint `/search`. Το request δηλώνει ότι το περιεχόμενο είναι JSON μέσω του `Content-Type`, ενώ το `prompt` μετατρέπεται σε JSON string με `JSON.stringify(prompt)`. Το συγκεκριμένο endpoint είναι υπεύθυνο για την επεξεργασία και validation των απαιτήσεων του χρήστη.

```
const response = await fetch("http://127.0.0.1:8000/search", {
  method: "POST",
  headers: {
    "Content-Type": "application/json",
  },
  body: JSON.stringify(prompt),
});
const requirements = await response.json();
```

Η χρήση του `await` σημαίνει ότι η function περιμένει να ολοκληρωθεί το HTTP request πριν συνεχίσει στο επόμενο βήμα. Όταν η απάντηση ληφθεί, μετατρέπεται από JSON σε JavaScript object μέσω της `response.json()`. Το Frontend ελέγχει στη συνέχεια το πεδίο `legit_prompt`, το οποίο έχει παραχθεί από το Backend. Αν το `legit_prompt` είναι `false`, το αίτημα θεωρείται μη έγκυρο και το `validation_message` εμφανίζεται στον χρήστη. Σε αυτή την περίπτωση δεν συνεχίζεται η αναζήτηση κατοικιών. Αντίθετα, αν είναι `true`, το Frontend προχωρά στο επόμενο στάδιο και ενημερώνει την κατάσταση εκτέλεσης ώστε να δείχνει ότι ξεκινά η αναζήτηση ακινήτων.

Μετά τον επιτυχή έλεγχο των απαιτήσεων, αποστέλλεται δεύτερο request προς το endpoint **`/search-properties`**. Το συγκεκριμένο request ενεργοποιεί τη διαδικασία αναζήτησης και αρχικού filtering των αγγελιών στο Backend. Η απάντηση είναι μία boolean τιμή που δηλώνει αν εντοπίστηκαν κατάλληλες κατοικίες. Αν η τιμή είναι `false`, η εφαρμογή εμφανίζει σχετικό μήνυμα στην περιοχή κατάστασης εκτέλεσης. Αν όμως υπάρχουν κατάλληλες αγγελίες, το Frontend ενημερώνει τον χρήστη ότι ξεκινά η δημιουργία των τελικών προτάσεων.

Όταν υπάρχουν υποψήφιες κατοικίες, πραγματοποιείται τρίτο request προς το endpoint **`/select-properties`**. Αυτό είναι το στάδιο στο οποίο το Backend καλεί το Recommendation / Ranking component και επιστρέφει τις τελικές προτεινόμενες κατοικίες. Η απάντηση μετατρέπεται και πάλι από JSON και αποθηκεύεται στο `recommendations state` μέσω της

setRecommendations(selectedProperties). Από τη στιγμή που ενημερώνεται το συγκεκριμένο state, το React ενημερώνει αυτόματα το σχετικό τμήμα του UI και δημιουργεί τις κάρτες των προτεινόμενων κατοικιών.

```
const selectedPropertiesResponse = await fetch(
  "http://127.0.0.1:8000/select-properties",
  {
    method: "POST",
  }
);
```

```
const selectedProperties = await selectedPropertiesResponse.json();
setRecommendations(selectedProperties);
```

Η συνολική διαδικασία των requests βρίσκεται μέσα σε try, catch και finally. Το try περιλαμβάνει τη βασική ροή εκτέλεσης, ενώ το catch ενεργοποιείται αν προκύψει κάποιο exception, για παράδειγμα αν το Backend δεν είναι διαθέσιμο ή αποτύχει κάποιο network request. Σε αυτή την περίπτωση το Frontend εμφανίζει γενικό μήνυμα σφάλματος και καθαρίζει την τρέχουσα κατάσταση. Το finally εκτελείται ανεξάρτητα από το αν η διαδικασία ολοκληρώθηκε επιτυχώς ή απέτυχε και θέτει το loading ξανά σε false, ώστε το κουμπί αναζήτησης να γίνει ξανά διαθέσιμο.

Στο δεύτερο βασικό τμήμα του App.jsx βρίσκεται το JSX, δηλαδή η περιγραφή της δομής της διεπαφής. Το JSX έχει σύνταξη παρόμοια με HTML, αλλά επιτρέπει την ενσωμάτωση JavaScript expressions μέσα στη δομή της σελίδας. Για παράδειγμα, το textarea χρησιμοποιεί το prompt ως value και κάθε αλλαγή που κάνει ο χρήστης ενημερώνει το state μέσω της setPrompt(). Με αυτόν τον τρόπο το πεδίο είναι controlled component, δηλαδή η τιμή που εμφανίζεται στο input ελέγχεται από το React state.

```
<textarea
  className="search-textarea"
  value={prompt}
  onChange={(event) => setPrompt(event.target.value)}
  placeholder="π.χ. Ψάχνω κατοικία στη Σητεία..."
/>
```

Αντίστοιχα, το κουμπί αναζήτησης συνδέεται με τη handleSearch() μέσω του onClick. Η ιδιότητα disabled={loading} σημαίνει ότι το κουμπί απενεργοποιείται

όσο βρίσκεται σε εξέλιξη η αναζήτηση, ενώ το κείμενό του αλλάζει δυναμικά από «Αναζήτηση» σε «Επεξεργασία...». Έτσι, το state δεν χρησιμοποιείται μόνο για την αποθήκευση δεδομένων αλλά επηρεάζει άμεσα τη συμπεριφορά και την εμφάνιση του UI.

```
<button
  className="search-button"
  onClick={handleSearch}
  disabled={loading}
>
  {loading ? "Επεξεργασία..." : "Αναζήτηση"}
</button>
```

Η εμφάνιση των validation messages γίνεται επίσης δυναμικά. Το expression {message && (...)} σημαίνει ότι το συγκεκριμένο τμήμα της διεπαφής εμφανίζεται μόνο όταν το message έχει περιεχόμενο. Επιπλέον, χρησιμοποιείται το isValid για να επιλεγεί διαφορετικό CSS class ανάλογα με το αν το μήνυμα αφορά επιτυχία ή σφάλμα. Με αυτόν τον τρόπο το ίδιο JSX block μπορεί να χρησιμοποιηθεί για διαφορετικές καταστάσεις χωρίς να απαιτούνται ξεχωριστά στοιχεία.

Το σημαντικότερο δυναμικό τμήμα του JSX αφορά την παρουσίαση των προτεινόμενων κατοικιών. Αν το recommendations array είναι κενό, εμφανίζεται ένα placeholder message που ενημερώνει τον χρήστη ότι οι προτάσεις θα εμφανιστούν μετά την ολοκλήρωση της αναζήτησης. Αν υπάρχουν αποτελέσματα, χρησιμοποιείται η JavaScript map() για να δημιουργηθεί μία κάρτα για κάθε αντικείμενο της λίστας. Σε κάθε iteration λαμβάνεται το item.property, το οποίο περιέχει τα structured στοιχεία της κατοικίας, ενώ το item.opinion.opinion περιέχει την αξιολόγηση που έχει παραχθεί από το LLM.

```
{recommendations.map((item, index) => {
  const property = item.property;

  return (
    <article
      className="recommendation-card"
      key={property.url}
    >
      ...
```



```
</article>  
);  
}}}
```

Το `index` χρησιμοποιείται για την εμφάνιση της θέσης της κατοικίας στη λίστα με τη μορφή #1, #2 κ.λπ., ενώ το `property.url` χρησιμοποιείται ως `key`, ώστε το React να μπορεί να αναγνωρίζει μοναδικά κάθε στοιχείο της λίστας. Μέσα σε κάθε `card` εμφανίζονται μόνο τα διαθέσιμα δεδομένα. Για παράδειγμα, το `location` εμφανίζεται μόνο αν υπάρχει τιμή, ενώ για αριθμητικά πεδία όπως `price` και `area_sqm` γίνεται έλεγχος με `!= null`. Αυτό αποτρέπει την εμφάνιση κενών ή μη έγκυρων στοιχείων όταν κάποια πληροφορία δεν έχει εξαχθεί από την αγγελία.

Τέλος, κάθε `recommendation card` εμφανίζει την περιγραφή της κατοικίας, την αξιολόγηση του LLM και σύνδεσμο προς την αρχική αγγελία στη ΧΕ. Ο σύνδεσμος ανοίγει σε νέο `tab` μέσω `target="_blank"`, ενώ το `rel="noopener noreferrer"` χρησιμοποιείται ως μέτρο ασφάλειας κατά το άνοιγμα εξωτερικών σελίδων.

Συνολικά, το `App.jsx` αποτελεί το σημείο στο οποίο συνδέονται η αλληλεπίδραση του χρήστη, η αποθήκευση της κατάστασης του Frontend, η επικοινωνία με τα API endpoints και η δυναμική παρουσίαση των αποτελεσμάτων. Με αυτόν τον τρόπο λειτουργεί ως το κεντρικό component του client-side μέρους της εφαρμογής και υλοποιεί πρακτικά ολόκληρη τη ροή που βλέπει και χειρίζεται ο χρήστης.

main.py

Το αρχείο **main.py** αποτελεί το κεντρικό αρχείο του Backend του SitiaHome. Σε αυτό δημιουργείται η εφαρμογή FastAPI, ορίζονται τα διαθέσιμα API endpoints και συντονίζεται η επικοινωνία μεταξύ του Frontend και των επιμέρους backend components. Το `main.py` δεν περιλαμβάνει την αναλυτική λογική επεξεργασίας απαιτήσεων, αναζήτησης ακινήτων ή recommendation/ranking. Αντίθετα, εισάγει τις αντίστοιχες functions από τα αρχεία των components και τις καλεί όταν λαμβάνονται τα κατάλληλα HTTP requests. Με αυτόν τον τρόπο λειτουργεί κυρίως ως orchestration layer,

δηλαδή ως το σημείο που ελέγχει τη σειρά με την οποία ενεργοποιούνται τα διαφορετικά τμήματα της εφαρμογής.

Στην αρχή του αρχείου γίνονται τα απαραίτητα imports. Από το FastAPI εισάγονται τα FastAPI και Body, ενώ το CORSMiddleware χρησιμοποιείται για τη ρύθμιση της επικοινωνίας με το Frontend. Παράλληλα, εισάγονται οι βασικές functions και models από τα υπόλοιπα backend modules: η search_xe_properties() από το prop_search.py, η process_and_validate_requirements() και το UserRequirements από το req_and_valid.py, καθώς και η select_properties() από το rank_and_rec.py.

Στη συνέχεια ορίζονται οι μεταβλητές matching_urls και user_requirements, οι οποίες χρησιμοποιούνται για την προσωρινή διατήρηση δεδομένων μεταξύ των διαδοχικών API calls της ίδιας εκτέλεσης. Το user_requirements αποθηκεύει τις δομημένες απαιτήσεις που προκύπτουν από το prompt, ενώ το matching_urls τη λίστα των URLs που πέρασαν το αρχικό filtering. Οι μεταβλητές αυτές βρίσκονται σε module-level scope και γι' αυτό χρησιμοποιείται το global μέσα στα endpoints όταν πρόκειται να τροποποιηθούν. Η συγκεκριμένη λύση είναι απλή και κατάλληλη για την τοπική, single-user μορφή του MVP, καθώς τα δεδομένα διατηρούνται μόνο προσωρινά στη μνήμη και δεν αποθηκεύονται σε database.

Με τη δημιουργία του app = FastAPI() αρχικοποιείται η FastAPI εφαρμογή. Αμέσως μετά προστίθεται το CORS middleware, ώστε να επιτρέπεται στο Frontend, το οποίο εκτελείται στο http://localhost:5173, να στέλνει HTTP requests προς το Backend. Επειδή Frontend και Backend λειτουργούν σε διαφορετικά origins, ο browser θα μπορούσε να απορρίψει την επικοινωνία λόγω της Same-Origin Policy. Το allow_origins περιορίζει την πρόσβαση στο συγκεκριμένο Frontend origin, ενώ τα allow_methods=["*"] και allow_headers=["*"] επιτρέπουν τα HTTP methods και headers που απαιτούνται από τα requests της εφαρμογής.

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:5173"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

Το πρώτο endpoint είναι το POST /search και αποτελεί το σημείο εισόδου του prompt στο Backend. Η παράμετρος prompt: str = Body(...) δηλώνει ότι το endpoint αναμένει ένα υποχρεωτικό string μέσα στο request body. Όταν το Frontend αποστέλλει το prompt, η function search() καλεί την process_and_validate_requirements(prompt). Η function αυτή αναλαμβάνει μέσω του LLM να μετατρέψει το φυσικό κείμενο σε structured UserRequirements και παράλληλα να ελέγξει αν το αίτημα είναι έγκυρο. Το αποτέλεσμα αποθηκεύεται στη μεταβλητή user_requirements και επιστρέφεται στο Frontend, όπου εξετάζεται το πεδίο legit_prompt.

```
@app.post("/search")
def search(prompt: str = Body(...)):
    global user_requirements

    user_requirements = process_and_validate_requirements(prompt)

    return user_requirements
```

Αν το αίτημα είναι έγκυρο, το Frontend καλεί στη συνέχεια το POST /search-properties. Το endpoint αυτό χρησιμοποιεί τα user_requirements που έχουν ήδη προκύψει από το προηγούμενο στάδιο και τα περνά στη search_xe_properties(). Η function πραγματοποιεί την αναζήτηση στην ΧΕ και επιστρέφει τα URLs των αγγελιών που ικανοποιούν τα βασικά κριτήρια. Τα URLs αποθηκεύονται στο matching_urls. Το endpoint δεν επιστρέφει ολόκληρη τη λίστα στο Frontend, αλλά μόνο boolean αποτέλεσμα: True όταν έχει βρεθεί τουλάχιστον μία κατάλληλη αγγελία και False όταν η λίστα είναι κενή. Με βάση αυτή την απάντηση, το Frontend αποφασίζει αν πρέπει να προχωρήσει στο επόμενο στάδιο.

```
@app.post("/search-properties")
def search_properties():
    global matching_urls

    matching_urls = search_xe_properties(user_requirements)

    if matching_urls:
        return True

    return False
```

Το τελευταίο endpoint είναι το POST /select-properties, το οποίο ενεργοποιεί το Recommendation / Ranking Engine. Εδώ χρησιμοποιούνται τόσο η λίστα matching_urls όσο και οι ήδη αποθηκευμένες user_requirements, ώστε η select_properties() να μπορεί να ανακτήσει τα αναλυτικά στοιχεία των υποψήφιων αγγελιών, να παράγει αξιολόγηση για κάθε κατοικία και, όταν υπάρχουν περισσότερες από πέντε, να επιλέξει και να κατατάξει τις πέντε καταλληλότερες. Η function είναι asynchronous, γι' αυτό και το endpoint ορίζεται με async def και η κλήση πραγματοποιείται με await. Το τελικό αποτέλεσμα αποθηκεύεται στο selected_properties και επιστρέφεται στο Frontend, όπου μετατρέπεται σε recommendation cards.

```
@app.post("/select-properties")
async def select_properties_main():
    selected_properties = await select_properties(matching_urls,user_requirements)

    return selected_properties
```

Συνολικά, το main.py υλοποιεί το κεντρικό API layer του SitiaHome και οργανώνει τη βασική ακολουθία λειτουργίας του Backend. Το /search χειρίζεται την επεξεργασία και validation του prompt, το /search-properties ενεργοποιεί την αναζήτηση και το αρχικό filtering των αγγελιών και το /select-properties εκτελεί την τελική αξιολόγηση και επιλογή των κατοικιών. Με αυτόν τον διαχωρισμό, το main.py παραμένει σχετικά μικρό και δεν περιέχει τη λεπτομερή business logic των επιμέρους διαδικασιών, αλλά λειτουργεί ως ο συνδετικός κρίκος που επιτρέπει στο Frontend και στα διαφορετικά backend modules να συνεργάζονται μέσα από μία οργανωμένη API-based ροή.

req_and_valid.py

Το αρχείο **req_and_valid.py** είναι υπεύθυνο για δύο άμεσα συνδεδεμένες λειτουργίες του SitiaHome: την επεξεργασία του prompt του χρήστη και τον έλεγχο εγκυρότητας των απαιτήσεων που εξάγονται από αυτό. Ουσιαστικά, παίρνει το ελεύθερο κείμενο που έχει εισάγει ο χρήστης και το μετατρέπει σε μία προκαθορισμένη structured μορφή, ώστε τα επόμενα components να μπορούν να χρησιμοποιήσουν συγκεκριμένα και σταθερά πεδία αντί για μη δομημένο φυσικό κείμενο. Παράλληλα, στο ίδιο LLM call πραγματοποιείται και validation, ώστε να καθοριστεί αν το αίτημα είναι επαρκές και κατάλληλο για να συνεχιστεί η διαδικασία αναζήτησης.

Στην αρχή του αρχείου φορτώνονται οι απαραίτητες βιβλιοθήκες και αρχικοποιείται ο OpenAI client. Η load_dotenv() χρησιμοποιείται για να φορτώσει τις μεταβλητές περιβάλλοντος από το αρχείο .env, όπως το OpenAI API key, ενώ το OpenAI() δημιουργεί τον client μέσω του οποίου πραγματοποιούνται τα requests προς το μοντέλο. Παράλληλα, από την Pydantic εισάγονται τα BaseModel και Field, τα οποία χρησιμοποιούνται για τον ορισμό του structured schema που πρέπει να επιστρέψει το LLM.

```
from dotenv import load_dotenv
from openai import OpenAI
from pydantic import BaseModel, Field
```

```
load_dotenv()
client = OpenAI()
```

Το βασικό structured model του αρχείου είναι το UserRequirements, το οποίο κληρονομεί από το BaseModel της Pydantic. Το model αυτό καθορίζει τα πεδία που πρέπει να περιλαμβάνει το αποτέλεσμα της επεξεργασίας. Το legit_prompt είναι boolean και δηλώνει αν το αίτημα θεωρείται έγκυρο. Το min_rent είναι προαιρετικό και μπορεί να είναι None όταν ο χρήστης δεν έχει δώσει ελάχιστο ενοίκιο, ενώ τα max_rent και min_bedrooms είναι τα δύο βασικά στοιχεία που απαιτούνται λογικά για έγκυρο αίτημα. Το additional_information συγκεντρώνει τις υπόλοιπες προτιμήσεις του χρήστη, όπως περιοχή, parking, επιπλωμένη κατοικία ή άλλα χαρακτηριστικά, ενώ το validation_message χρησιμοποιείται για να επιστρέψει είτε "OK" είτε σύντομη αιτιολόγηση όταν το αίτημα είναι μη έγκυρο.

```
class UserRequirements(BaseModel):
    legit_prompt: bool
```

```
min_rent: int | None
max_rent: int | None
min_bedrooms: int | None
additional_information: str | None
validation_message: str
```

Τα Field(...) descriptions που υπάρχουν στην πραγματική υλοποίηση λειτουργούν ως επιπλέον οδηγίες για το structured output. Δηλαδή, δεν καθορίζουν μόνο τον τύπο κάθε πεδίου, αλλά εξηγούν και τη σημασία του στο μοντέλο. Αυτό βοηθά το LLM να γνωρίζει τι ακριβώς πρέπει να τοποθετήσει σε κάθε attribute. Για παράδειγμα, στο min_rent δηλώνεται ρητά ότι είναι optional και ότι πρέπει να επιστρέφεται null όταν ο χρήστης δεν έχει καθορίσει ελάχιστο ενοίκιο. Με αυτόν τον τρόπο αποφεύγεται η αυθαίρετη δημιουργία τιμών που δεν έχουν αναφερθεί από τον χρήστη.

Το επόμενο σημαντικό τμήμα είναι το system prompt REQUIREMENTS_PROCESSING_AND_VALIDATION_LLM_SYSTEM_PROMPT. Εκεί περιγράφεται αναλυτικά η συμπεριφορά που πρέπει να ακολουθήσει το LLM. Καθορίζεται ότι τα υποχρεωτικά πεδία είναι το max_rent και το min_bedrooms, ενώ το min_rent παραμένει προαιρετικό. Αν ο χρήστης δώσει εύρος τιμής, η χαμηλότερη τιμή αποθηκεύεται ως min_rent και η υψηλότερη ως max_rent. Αν δεν δοθεί ελάχιστο ενοίκιο, επιστρέφεται null. Επιπλέον, το prompt ζητά να μην εφευρίσκονται τιμές, να μην επαναλαμβάνονται τα βασικά πεδία μέσα στο additional_information και να αποθηκεύονται εκεί μόνο οι υπόλοιπες προτιμήσεις.

Required fields:

- max_rent
- min_bedrooms

Rules:

- If the user gives a rent range, use the lower value as min_rent and the upper value as max_rent.
- If no minimum rent is specified, return null for min_rent.
- Do not invent missing values.

Στο ίδιο system prompt ορίζονται και οι κανόνες του validation. Το legit_prompt πρέπει να γίνει false όταν λείπει το max_rent ή το min_bedrooms, όταν το αίτημα δεν αφορά μακροχρόνια ενοικίαση στον Δήμο Σητείας ή όταν κάποια τιμή είναι μη έγκυρη. Υπάρχουν επίσης συγκεκριμένοι περιορισμοί, όπως η απαγόρευση αρνητικών τιμών ενοικίου, η απαίτηση το max_rent να μην είναι

μικρότερο από το `min_rent` όταν το τελευταίο έχει δηλωθεί και ο περιορισμός του `min_bedrooms` σε ακέραιο από 0 έως 9. Για `studio` ή γκαρσονιέρα, το `min_bedrooms` ορίζεται σε 0. Παράλληλα, αν ο χρήστης δεν αναφέρει τοποθεσία ή διάρκεια μίσθωσης, το σύστημα θεωρεί ως default τον Δήμο Σητείας και τη μακροχρόνια μίσθωση αντίστοιχα. Αν το αίτημα είναι έγκυρο, το `validation_message` παίρνει την τιμή "OK", διαφορετικά επιστρέφεται σύντομο μήνυμα στα ελληνικά.

Η βασική function του αρχείου είναι η `process_and_validate_requirements()`, η οποία δέχεται ως είσοδο το αρχικό `user_prompt` σε μορφή `string` και πραγματοποιεί το LLM call. Μέσω της `client.responses.parse()` χρησιμοποιείται το μοντέλο `gpt-5.6-luna`, ενώ στο `instructions` περνά το `system prompt` και στο `input` το κείμενο του χρήστη. Το σημαντικότερο στοιχείο είναι το `text_format=UserRequirements`, το οποίο ζητά από το μοντέλο να επιστρέψει `structured output` που να ακολουθεί ακριβώς το Pydantic schema που έχει οριστεί.

```
def process_and_validate_requirements(user_prompt:str)-> UserRequirements:
```

```
    response = client.responses.parse(
        model="gpt-5.6-luna",
        instructions= REQUIREMENTS_PROCESSING_AND_VALIDATION_LLM_SYSTEM_PROMPT,
        input=user_prompt,
        text_format=UserRequirements,
    )

    return response.output_parsed
```

Η `response.output_parsed` περιέχει ήδη το αποτέλεσμα ως αντικείμενο τύπου `UserRequirements`, επομένως δεν απαιτείται χειροκίνητο parsing JSON ή επιπλέον μετατροπή των δεδομένων. Για παράδειγμα, ένα αίτημα όπως «Ψάχνω σπίτι στη Σητεία έως 450 ευρώ με τουλάχιστον ένα υπνοδωμάτιο και ιδανικά parking» μπορεί να μετατραπεί σε `structured` μορφή με `max_rent=450`, `min_bedrooms=1`, `min_rent=None`, τις επιπλέον προτιμήσεις στο `additional_information` και `legit_prompt=True`. Αυτό το αντικείμενο επιστρέφεται στη συνέχεια στο `main.py`, αποθηκεύεται προσωρινά στο `user_requirements` και χρησιμοποιείται τόσο για να ενημερωθεί το Frontend

για το αποτέλεσμα του validation όσο και από τα επόμενα backend components.

Συνολικά, το `req_and_valid.py` λειτουργεί ως η γέφυρα μεταξύ της φυσικής γλώσσας του χρήστη και της οργανωμένης μορφής δεδομένων που χρειάζεται η εφαρμογή. Συνδυάζοντας LLM processing, Pydantic structured outputs και validation rules, επιτρέπει στο SitiaHome να δέχεται ευέλικτα prompts χωρίς να απαιτεί από τον χρήστη να συμπληρώνει προκαθορισμένα fields, ενώ ταυτόχρονα εξασφαλίζει ότι τα επόμενα στάδια λαμβάνουν ένα σταθερό και ελεγχόμενο schema. Έτσι, η πολυπλοκότητα της κατανόησης του φυσικού κειμένου περιορίζεται σε ένα συγκεκριμένο component και το υπόλοιπο σύστημα μπορεί να λειτουργεί πάνω σε οργανωμένα δεδομένα.

prop_search.py

Το αρχείο **prop_search.py** είναι υπεύθυνο για το στάδιο της αναζήτησης και του αρχικού filtering των διαθέσιμων αγγελιών από τη Χρυσή Ευκαιρία (ΧΕ). Δέχεται ως είσοδο τις ήδη επεξεργασμένες και επικυρωμένες απαιτήσεις του χρήστη και εντοπίζει ποιες αγγελίες ικανοποιούν τα βασικά υποχρεωτικά κριτήρια πριν περάσουν στο επόμενο στάδιο του Recommendation / Ranking. Ο ρόλος του είναι σκόπιμα περιορισμένος: δεν πραγματοποιεί πλήρη αξιολόγηση κάθε ακινήτου και δεν χρησιμοποιεί LLM, αλλά εκτελεί μία πιο απλή και deterministic διαδικασία ανάκτησης δεδομένων, parsing και filtering. Με αυτόν τον τρόπο μειώνεται ο αριθμός των αγγελιών που χρειάζεται να υποστούν αργότερα πιο σύνθετη και ακριβή επεξεργασία.

Στην αρχή του αρχείου εισάγονται οι βιβλιοθήκες που απαιτούνται για την ανάκτηση και επεξεργασία των δεδομένων. Η `httpx` χρησιμοποιείται για την πραγματοποίηση HTTP request προς τη σελίδα αποτελεσμάτων της ΧΕ, η `BeautifulSoup` για την ανάλυση του HTML, ενώ οι `html` και `json` χρησιμοποιούνται για την αποκωδικοποίηση και μετατροπή των structured δεδομένων που βρίσκονται ενσωματωμένα στη σελίδα. Εισάγεται επίσης το `UserRequirements` από το `req_and_valid.py`, ώστε οι functions του αρχείου να γνωρίζουν τη δομή των απαιτήσεων που λαμβάνουν ως είσοδο.

```
import html
import json
import httpx
from bs4 import BeautifulSoup
from components.req_and_valid import UserRequirements
```


Στη συνέχεια ορίζεται η σταθερά `XE_RESULTS_URL`, η οποία περιέχει τη διεύθυνση της σελίδας αποτελεσμάτων της ΧΕ για ενοικίαση κατοικιών στην επιλεγμένη γεωγραφική περιοχή. Στο URL έχουν ήδη ενσωματωθεί βασικές παράμετροι, όπως ότι πρόκειται για συναλλαγή τύπου `rent`, ότι το είδος ακινήτου είναι `residence` και τα γεωγραφικά όρια της περιοχής αναζήτησης. Με αυτόν τον τρόπο το `component` δεν ξεκινά από ολόκληρη τη βάση αγγελιών της ΧΕ, αλλά από μία ήδη περιορισμένη σελίδα αποτελεσμάτων που αφορά τον Δήμο Σητείας και κατοικίες προς ενοικίαση.

Η `function fetch_xe_json()` αναλαμβάνει την ανάκτηση της σελίδας αποτελεσμάτων. Δημιουργείται ένα HTTP header με User-Agent που προσομοιώνει έναν κανονικό desktop browser και στη συνέχεια πραγματοποιείται GET request με την `httpx.get()`. Η επιλογή `follow_redirects=True` επιτρέπει στην request να ακολουθήσει πιθανές ανακατευθύνσεις, ενώ το `timeout=30.0` θέτει ανώτατο χρονικό όριο αναμονής. Με το `response.raise_for_status()` προκαλείται exception αν η ΧΕ επιστρέψει HTTP error status, ώστε να μην συνεχιστεί η επεξεργασία με μη έγκυρη απόκριση.

```
response = httpx.get(
    XE_RESULTS_URL,
    headers=headers,
    follow_redirects=True,
    timeout=30.0,
)
```

```
response.raise_for_status()
```

Αφού ληφθεί το HTML της σελίδας, χρησιμοποιείται η BeautifulSoup για να εντοπιστεί το div με `id="application_container"`. Η ΧΕ αποθηκεύει μέσα στο attribute `data-json-data structured` πληροφορία για τα αποτελέσματα της αναζήτησης. Η τιμή αυτή εξάγεται, αποκωδικοποιείται με `html.unescape()` και στη συνέχεια μετατρέπεται από JSON string σε Python dictionary με `json.loads()`. Έτσι αποφεύγεται η ανάγκη να γίνει scraping κάθε ορατού HTML element ξεχωριστά, καθώς το `component` αξιοποιεί τα ήδη οργανωμένα δεδομένα που υπάρχουν στη σελίδα.

```
soup = BeautifulSoup(response.text, "html.parser")
container = soup.find("div", id="application_container")
```

```
raw_json = container.get("data-json-data")
```

```
decoded_json = html.unescape(raw_json)
data = json.loads(decoded_json)
```

Η function `parse_price()` είναι μία μικρή βοηθητική function που μετατρέπει την τιμή μιας αγγελίας από string σε integer. Η τιμή που επιστρέφεται από την ΧΕ μπορεί να περιλαμβάνει το σύμβολο €, τελείες ως separators χιλιάδων, non-breaking spaces και επιπλέον κενά. Η function αφαιρεί αυτά τα στοιχεία και στο τέλος μετατρέπει το καθαρισμένο string σε ακέραιο αριθμό. Αυτό είναι απαραίτητο ώστε η τιμή να μπορεί να συγκριθεί αριθμητικά με τα `min_rent` και `max_rent` του χρήστη.

```
def parse_price(price: str) -> int:
    return int(
        price
        .replace("€", "")
        .replace("\xa0", "")
        .replace(".", "")
        .strip()
    )
```

Η βασική λογική filtering βρίσκεται στη function `filter_xe_listings()`. Η function δέχεται δύο inputs: το data dictionary που προέκυψε από την ΧΕ και ένα αντικείμενο `UserRequirements`. Δημιουργείται αρχικά μία κενή λίστα `matching_urls` και στη συνέχεια γίνεται iteration σε κάθε αγγελία που βρίσκεται στο `data["results"]`. Για κάθε listing εξάγεται η τιμή και ο αριθμός υπνοδωματίων. Αν η ΧΕ δεν έχει διαθέσιμη τιμή για τα bedrooms και επιστρέψει `None`, ο κώδικας θεωρεί την τιμή ως 0, ώστε η σύγκριση με το `min_bedrooms` να μπορεί να πραγματοποιηθεί χωρίς error.

```
for listing in data["results"]:
    price = parse_price(listing["price"])
    bedrooms = listing["bedrooms"]

    if bedrooms is None:
        bedrooms = 0
```

Στη συνέχεια εφαρμόζονται τα βασικά κριτήρια του χρήστη. Η τιμή του ακινήτου πρέπει να είναι μικρότερη ή ίση από το `max_rent`, ενώ τα bedrooms πρέπει να είναι τουλάχιστον ίσα με το `min_bedrooms`. Το `min_rent` είναι

optional, επομένως ο σχετικός έλεγχος γίνεται μόνο όταν υπάρχει τιμή. Η συνθήκη (`requirements.min_rent is None or price >= requirements.min_rent`) σημαίνει ότι αν ο χρήστης δεν έχει δηλώσει κατώτατο όριο, το συγκεκριμένο μέρος του filtering θεωρείται αυτόματα ικανοποιημένο. Αν όλα τα κριτήρια είναι αληθή, στη λίστα αποθηκεύεται μόνο το URL της αγγελίας.

```
if (  
    (requirements.min_rent is None or price >= requirements.min_rent)  
    and price <= requirements.max_rent  
    and bedrooms >= requirements.min_bedrooms  
):  
    matching_urls.append(listing["url"])
```

Το γεγονός ότι το `prop_search.py` επιστρέφει μόνο URLs είναι σημαντικό για τη συνολική αρχιτεκτονική. Σε αυτό το στάδιο δεν υπάρχει ανάγκη να ανακτηθούν αναλυτικά δεδομένα από κάθε επιμέρους αγγελία, όπως πλήρης περιγραφή, τετραγωνικά μέτρα ή κατάσταση επίπλωσης. Το `component` εκτελεί μόνο το πρώτο, γρήγορο filtering με βάση τα `hard criteria`. Έτσι, μόνο οι αγγελίες που έχουν ήδη περάσει αυτόν τον έλεγχο θα ανοίξουν αργότερα από το `rank_and_rec.py`, όπου πραγματοποιείται η πιο σύνθετη επεξεργασία με LLM. Αυτό μειώνει τον αριθμό των HTTP requests και των LLM calls που απαιτούνται.

Η τελευταία function του αρχείου είναι η `search_xe_properties()`, η οποία λειτουργεί ως η κύρια public function του `component`. Δέχεται ένα `UserRequirements`, καλεί πρώτα τη `fetch_xe_json()` για να ανακτήσει τα δεδομένα της ΧΕ και στη συνέχεια περνά τα δεδομένα και τις απαιτήσεις στη `filter_xe_listings()`. Το αποτέλεσμα είναι μία λίστα από URLs που αντιστοιχούν στις αγγελίες που πέρασαν τα βασικά κριτήρια.

```
def search_xe_properties(requirements: UserRequirements,) -> list[str]:  
  
    data = fetch_xe_json()  
    matching_urls = filter_xe_listings(data, requirements)  
    return matching_urls
```

Η συγκεκριμένη function καλείται από το endpoint `/search-properties` του `main.py`. Η λίστα που επιστρέφεται αποθηκεύεται προσωρινά στο `matching_urls`. Αν η λίστα περιέχει τουλάχιστον ένα URL, το Backend ενημερώνει το Frontend ότι βρέθηκαν υποψήφιες κατοικίες και η διαδικασία συνεχίζεται προς το Recommendation / Ranking Engine. Αν είναι κενή, η ροή

σταματά και ο χρήστης ενημερώνεται ότι δεν εντοπίστηκαν ακίνητα που να ικανοποιούν τα βασικά κριτήριά του.

Συνολικά, το `prop_search.py` λειτουργεί ως το πρώτο φίλτρο ανάμεσα στις διαθέσιμες αγγελίες της ΧΕ και στο πιο απαιτητικό recommendation στάδιο. Η υλοποίησή του συνδυάζει HTTP requests, HTML parsing, αξιοποίηση embedded JSON δεδομένων και deterministic filtering με βάση τις δομημένες απαιτήσεις του χρήστη. Ο διαχωρισμός αυτός είναι σημαντικός, επειδή απλά και σαφώς ορισμένα κριτήρια, όπως η τιμή και ο αριθμός υπνοδωματίων, ελέγχονται προγραμματιστικά χωρίς χρήση AI, ενώ οι πιο σύνθετες και υποκειμενικές προτιμήσεις αφήνονται για το επόμενο component.

rank_and_rec.py

Το αρχείο **rank_and_rec.py** είναι το πιο σύνθετο backend component του SitiaHome, γιατί αναλαμβάνει το στάδιο της αναλυτικής επεξεργασίας, αξιολόγησης και τελικής επιλογής των υποψήφιων κατοικιών που έχουν ήδη περάσει από το αρχικό filtering του `prop_search.py`. Δέχεται ως είσοδο τη λίστα `matching_urls` και τις δομημένες απαιτήσεις του χρήστη και, για κάθε αγγελία, ανακτά τα αναλυτικά δεδομένα της από τη ΧΕ, τα μετατρέπει σε κοινό structured schema μέσω LLM, δημιουργεί μία σύντομη αξιολόγηση για το πόσο καλά ταιριάζει η κατοικία στις ανάγκες του χρήστη και, αν οι διαθέσιμες κατοικίες είναι περισσότερες από πέντε, επιλέγει και κατατάσσει τις πέντε καταλληλότερες.

Στην αρχή του αρχείου φορτώνονται οι απαραίτητες βιβλιοθήκες και δημιουργείται ένας asynchronous OpenAI client με `AsyncOpenAI()`. Η χρήση asynchronous client είναι σημαντική επειδή το συγκεκριμένο component πραγματοποιεί πολλαπλά HTTP requests και LLM calls για διαφορετικές αγγελίες, διαδικασίες που μπορούν να εκτελεστούν παράλληλα ώστε να μειωθεί ο συνολικός χρόνος αναμονής. Παράλληλα χρησιμοποιούνται οι `httpx`, `asyncio`, `BeautifulSoup`, `json` και `html` για την ανάκτηση και επεξεργασία των δεδομένων των αγγελιών. Το `.env` φορτώνεται με `load_dotenv()`, ώστε ο OpenAI client να έχει πρόσβαση στο απαραίτητο API key.

```
load_dotenv()
client = AsyncOpenAI()
```

Στη συνέχεια ορίζονται τα Pydantic models που χρησιμοποιούνται ως structured schemas στα διαφορετικά στάδια της επεξεργασίας. Το PropertyListing περιγράφει την κοινή δομή μιας κατοικίας και περιλαμβάνει πεδία όπως τίτλο, τιμή, αριθμό υπνοδωματίων, περιοχή, τετραγωνικά μέτρα, περιγραφή, κατάσταση επίπλωσης και URL. Το PropertyListing_op περιέχει την αξιολόγηση που παράγεται από το LLM για μία κατοικία, ενώ το FinalProperty συνδυάζει τα structured στοιχεία της κατοικίας με αυτή την αξιολόγηση. Τέλος, το PropertySelection χρησιμοποιείται στο τελικό ranking stage και περιέχει μία ταξινομημένη λίστα από URLs.

```
class FinalProperty(BaseModel):  
    property: PropertyListing  
    opinion: PropertyListing_op
```

```
class PropertySelection(BaseModel):  
    urls: list[str]
```

Η function `extract_property_json()` αναλαμβάνει να εξαγάγει από το HTML μιας μεμονωμένης αγγελίας μόνο τα structured δεδομένα που αφορούν το συγκεκριμένο ακίνητο. Όπως και στο `prop_search.py`, αναζητείται το `div` με `id="application_container"` και το attribute `data-json-data`. Η τιμή αποκωδικοποιείται με `html.unescape()` και μετατρέπεται σε Python dictionary με `json.loads()`. Από το συνολικό JSON διατηρείται μόνο το πεδίο `"result"`, δηλαδή το τμήμα που αφορά την ίδια την αγγελία. Στη συνέχεια αυτό μετατρέπεται ξανά σε JSON string με `json.dumps()`, ώστε να μπορεί να σταλεί στο LLM. Η επιλογή να στέλνεται μόνο το `"result"` και όχι ολόκληρο το HTML μειώνει σημαντικά το μέγεθος του input και κατά συνέπεια το token usage.

```
property_data = json.loads(html.unescape(raw_json)).get("result")  
return json.dumps(property_data, ensure_ascii=False) if property_data else None
```

Η function `extract_property_with_llm()` δέχεται το structured JSON μιας αγγελίας και το URL της και χρησιμοποιεί το μοντέλο `gpt-5.4-mini` για να μετατρέψει τα δεδομένα σε αντικείμενο `PropertyListing`. Το prompt καθορίζει ότι το μοντέλο πρέπει να χρησιμοποιεί μόνο πληροφορίες που υπάρχουν στα δεδομένα, να μην εφευρίσκει τιμές και να επιστρέφει `null` όταν κάτι δεν μπορεί

να προσδιοριστεί αξιόπιστα. Μέσω του `text_format=PropertyListing`, το αποτέλεσμα επιστρέφεται απευθείας στη συγκεκριμένη Pydantic δομή. Έτσι, ακόμη και αν τα αρχικά δεδομένα της ΧΕ έχουν διαφορετική ή σύνθετη μορφή, το υπόλοιπο σύστημα δουλεύει πάνω σε ένα σταθερό schema.

Η function `get_properties_details()` αναλαμβάνει να εφαρμόσει αυτή τη διαδικασία σε όλα τα `matching_urls`. Δημιουργείται ένας `httpx.AsyncClient`, ώστε τα HTTP requests προς τις διαφορετικές αγγελίες να μπορούν να πραγματοποιηθούν *asynchronously*. Μέσα στη function ορίζεται η `process_property()`, η οποία για κάθε URL κατεβάζει τη σελίδα, εξάγει το property JSON και στη συνέχεια καλεί την `extract_property_with_llm()`. Το σημαντικότερο σημείο είναι η χρήση της `asyncio.gather()`, η οποία ξεκινά την επεξεργασία πολλών κατοικιών χωρίς να περιμένει να ολοκληρωθεί η μία πριν ξεκινήσει η επόμενη.

```
results = await asyncio.gather(  
    *(process_property(url) for url in matching_urls)  
)
```

Μετά την ανάκτηση των structured `PropertyListing` αντικειμένων, ξεκινά η κύρια function `select_properties()`. Αρχικά καλείται η `get_properties_details()` και, αν δεν έχει προκύψει καμία κατοικία, επιστρέφεται κενή λίστα. Στη συνέχεια ορίζεται η εσωτερική *asynchronous* function `generate_property_opinion()`, η οποία στέλνει στο LLM τόσο τις δομημένες απαιτήσεις του χρήστη όσο και τα δεδομένα κάθε κατοικίας. Το μοντέλο παράγει μία μεσαίου μήκους αξιολόγηση στα ελληνικά, η οποία μπορεί να είναι θετική, αρνητική ή μικτή και λαμβάνει υπόψη *strengths*, *weaknesses* και *trade-offs*. Η κατοικία και η αξιολόγησή της συνδυάζονται σε ένα `FinalProperty`.

```
return FinalProperty( property=property, opinion=response.output_parsed)
```

Οι αξιολογήσεις όλων των κατοικιών παράγονται επίσης παράλληλα μέσω `asyncio.gather()`. Αυτό είναι ιδιαίτερα χρήσιμο, επειδή κάθε *opinion* απαιτεί ξεχωριστό LLM call. Αν γίνονταν σειριακά, ο συνολικός χρόνος εκτέλεσης θα αυξανόταν σημαντικά. Από τα αποτελέσματα διατηρούνται μόνο όσα δεν είναι `None`, δηλαδή όσα ολοκληρώθηκαν επιτυχώς.

Αν μετά από αυτό το στάδιο υπάρχουν πέντε ή λιγότερες κατοικίες, η function τις επιστρέφει όλες χωρίς επιπλέον ranking:

```
if len(final_properties) <= 5:  
    return final_properties
```

Αν όμως υπάρχουν περισσότερες από πέντε, ενεργοποιείται ένα δεύτερο LLM-based στάδιο επιλογής. Όλα τα FinalProperty μετατρέπονται σε JSON και αποστέλλονται στο μοντέλο μαζί με τα user_requirements. Το μοντέλο καλείται να επιλέξει ακριβώς πέντε URLs και να τα επιστρέψει ταξινομημένα από το καλύτερο προς το ασθενέστερο match. Το text_format=PropertySelection εξασφαλίζει ότι η απάντηση θα περιέχει μία structured λίστα URLs.

Στη συνέχεια δημιουργείται ένα dictionary properties_by_url, ώστε κάθε URL να αντιστοιχεί στο αρχικό FinalProperty. Αυτό είναι σημαντικό γιατί το LLM επιστρέφει μόνο URLs και όχι ξανά ολόκληρα τα δεδομένα των κατοικιών. Η τελική λίστα δημιουργείται με βάση τη σειρά των URLs που επέστρεψε το LLM, διατηρώντας έτσι και το ranking.

```
properties_by_url = {
    property.property.url: property
    for property in final_properties
}

return [
    properties_by_url[url]
    for url in response.output_parsed.urls
    if url in properties_by_url
][:5]
```

Τέλος, σε αρκετά σημεία υπάρχουν try/except blocks ώστε μία αποτυχία σε HTTP request, parsing ή LLM call να μην διακόψει ολόκληρη τη διαδικασία. Αν για παράδειγμα αποτύχει η επεξεργασία μίας αγγελίας, αυτή μπορεί να αγνοηθεί και να συνεχιστεί η επεξεργασία των υπολοίπων. Αν αποτύχει το τελικό ranking call, ως fallback επιστρέφονται οι πρώτες πέντε διαθέσιμες κατοικίες.

Συνολικά, το rank_and_rec.py υλοποιεί το πιο AI-intensive στάδιο του SitiaHome. Συνδυάζει asynchronous HTTP requests, HTML/JSON extraction, Pydantic structured outputs, πολλαπλά LLM calls και τελικό ranking. Η βασική λογική είναι: ανάκτηση αναλυτικών δεδομένων → μετατροπή σε κοινό schema → παραγωγή αξιολόγησης για κάθε κατοικία → προαιρετικό top-5 ranking → επιστροφή FinalProperty αντικειμένων στο Backend. Έτσι, το component μετατρέπει μία απλή λίστα URLs σε ένα μικρό και οργανωμένο σύνολο προτεινόμενων κατοικιών, έτοιμο να παρουσιαστεί στο Frontend.

2.3.3 Παραδείγματα Εκτέλεσης και Συμπεριφοράς της Εφαρμογής

Στο υποκεφάλαιο αυτό θα παρουσιαστούν διαφορετικά παραδείγματα εκτέλεσης του SitiaHome, με στόχο να φανεί στην πράξη ο τρόπος με τον οποίο η εφαρμογή ανταποκρίνεται σε διαφορετικά σενάρια χρήσης. Θα εξεταστούν περιπτώσεις όπως ένα μη έγκυρο ή ελλιπές αίτημα, μία έγκυρη αναζήτηση στην οποία δεν εντοπίζονται κατάλληλες κατοικίες, καθώς και μία επιτυχημένη εκτέλεση όπου ολοκληρώνεται όλη η ροή και εμφανίζονται οι τελικές προτεινόμενες κατοικίες.

Μέσα από τα παραδείγματα αυτά θα γίνει πιο σαφής τόσο η συνολική συνεργασία των επιμέρους components όσο και ο τρόπος με τον οποίο το σύστημα διαχειρίζεται διαφορετικές πιθανές εκβάσεις κατά τη λειτουργία του.

Περίπτωση μη έγκυρου ή ελλιπούς αιτήματος

Περιγραφή Αναζήτησης

Ποια είναι η πρωτεύουσα της Ισπανίας;

Υποχρεωτικά στοιχεία: μέγιστο ενοίκιο και ελάχιστος αριθμός υπνοδωματίων

Αναζήτηση

ΚΑΤΑΣΤΑΣΗ ΕΚΤΕΛΕΣΗΣ

Το αίτημα δεν αφορά μακροχρόνια ενοικίαση κατοικίας στον Δήμο Σητείας και λείπουν τα απαιτούμενα στοιχεία.

Προτεινόμενες Κατοικίες

Οι προτεινόμενες κατοικίες θα εμφανιστούν εδώ μετά την ολοκλήρωση της αναζήτησης.

Στην εικόνα παρουσιάζεται ένα παράδειγμα μη έγκυρου αιτήματος χρήστη. Ο χρήστης έχει εισαγάγει την ερώτηση «Ποια είναι η πρωτεύουσα της Ισπανίας;», η οποία δεν σχετίζεται με αναζήτηση κατοικίας για μακροχρόνια ενοικίαση στον Δήμο Σητείας. Η εφαρμογή απορρίπτει το αίτημα κατά το στάδιο του Requirements Validation και εμφανίζει σχετικό μήνυμα σφάλματος με κόκκινη ένδειξη, ενημερώνοντας τον χρήστη για τον λόγο απόρριψης. Παράλληλα, δεν πραγματοποιείται αναζήτηση ακινήτων, το panel «Κατάσταση Εκτέλεσης» παραμένει χωρίς περαιτέρω ενημέρωση και η περιοχή «Προτεινόμενες Κατοικίες» δεν εμφανίζει αποτελέσματα. Έτσι φαίνεται στην πράξη ότι η εφαρμογή σταματά τη ροή όταν το prompt είναι εκτός σκοπού ή ελλιπές και δεν προχωρά άσκοπα στα επόμενα backend components.

🔍 Περιγραφή Αναζήτησης

Ψάχνω διαμέρισμα στη Σητεία με ενοίκιο έως 450 ευρώ. Ιδανικά, θέλω να είναι επιπλωμένο και να διαθέτει θέση στάθμευσης.

🕒 Υποχρεωτικά στοιχεία: μέγιστο ενοίκιο και ελάχιστος αριθμός υπνοδωματίων

Επεξεργασία...

ΚΑΤΑΣΤΑΣΗ ΕΚΤΕΛΕΣΗΣ

ΕΛΕΓΧΟΣ ΑΙΤΗΜΑΤΟΣ

Ελέγχεται το αίτημα που έχετε καταχωρήσει ως προς την εγκυρότητά του.

🏠 Προτεινόμενες Κατοικίες

Οι προτεινόμενες κατοικίες θα εμφανιστούν εδώ μετά την ολοκλήρωση της αναζήτησης.

🔍 Περιγραφή Αναζήτησης

Ψάχνω διαμέρισμα στη Σητεία με ενοίκιο έως 450 ευρώ. Ιδανικά, θέλω να είναι επιπλωμένο και να διαθέτει θέση στάθμευσης.

🕒 Υποχρεωτικά στοιχεία: μέγιστο ενοίκιο και ελάχιστος αριθμός υπνοδωματίων

Αναζήτηση

ΚΑΤΑΣΤΑΣΗ ΕΚΤΕΛΕΣΗΣ

❗ Παρακαλώ αναφέρετε τον ελάχιστο αριθμό υπνοδωματίων.

🏠 Προτεινόμενες Κατοικίες

Οι προτεινόμενες κατοικίες θα εμφανιστούν εδώ μετά την ολοκλήρωση της αναζήτησης.

Στην επάνω εικόνα φαίνεται το ενδιαμέσο στάδιο της εκτέλεσης, όπου το panel «Κατάσταση Εκτέλεσης» ενημερώνεται δυναμικά και εμφανίζει την ένδειξη «ΕΛΕΓΧΟΣ ΑΙΤΗΜΑΤΟΣ», μαζί με μήνυμα ότι το αίτημα ελέγχεται ως προς την εγκυρότητά του.

Περίπτωση έγκυρης αναζήτησης στην οποία δεν εντοπίζονται κατάλληλες κατοικίες

SitiaHome Εύρεση κατοικίας στον Δήμο Σητείας

🔍 Περιγραφή Αναζήτησης

Ψάχνω διαμέρισμα με δύο υπνοδωμάτια στη Σητεία με ενοίκιο έως 220 ευρώ. Ιδανικά, θέλω να είναι επιπλωμένο και να διαθέτει θέση στάθμευσης.

📌 Υποχρεωτικά στοιχεία: μέγιστο ενοίκιο και ελάχιστος αριθμός υπνοδωματίων

Επεξεργασία...

ΚΑΤΑΣΤΑΣΗ ΕΚΤΕΛΕΣΗΣ

ΑΝΑΖΗΤΗΣΗ ΑΚΙΝΗΤΩΝ

Έγκυρο Αίτημα. Πραγματοποιείται αναζήτηση ακινήτων βάσει των προτιμήσεων και απαιτήσεών σας.

🏠 Προτεινόμενες Κατοικίες

Οι προτεινόμενες κατοικίες θα εμφανιστούν εδώ μετά την ολοκλήρωση της αναζήτησης.

🔍 Περιγραφή Αναζήτησης

Ψάχνω διαμέρισμα με δύο υπνοδωμάτια στη Σητεία με ενοίκιο έως 220 ευρώ. Ιδανικά, θέλω να είναι επιπλωμένο και να διαθέτει θέση στάθμευσης.

📌 Υποχρεωτικά στοιχεία: μέγιστο ενοίκιο και ελάχιστος αριθμός υπνοδωματίων

Αναζήτηση

ΚΑΤΑΣΤΑΣΗ ΕΚΤΕΛΕΣΗΣ

ΔΕΝ ΒΡΕΘΗΚΑΝ ΑΚΙΝΗΤΑ

Δεν βρέθηκε κατοικία που να πληροί τις απαιτήσεις σας. Παρακαλώ υποβάλετε νέο αίτημα.

🏠 Προτεινόμενες Κατοικίες

Οι προτεινόμενες κατοικίες θα εμφανιστούν εδώ μετά την ολοκλήρωση της αναζήτησης.

Στις δύο πρώτες εικόνες παρουσιάζεται η εξέλιξη ενός έγκυρου αιτήματος κατά τα αρχικά στάδια της εκτέλεσης. Αρχικά, το panel «Κατάσταση Εκτέλεσης» εμφανίζει την ένδειξη «ΕΛΕΓΧΟΣ ΑΙΤΗΜΑΤΟΣ», καθώς το σύστημα επεξεργάζεται και ελέγχει αν το prompt περιλαμβάνει τις απαιτούμενες πληροφορίες. Αφού το αίτημα κριθεί έγκυρο, η κατάσταση αλλάζει σε «ΑΝΑΖΗΤΗΣΗ ΑΚΙΝΗΤΩΝ», γεγονός που δείχνει ότι η εφαρμογή έχει προχωρήσει στο Property Search και πραγματοποιεί αναζήτηση αγγελιών με βάση τα κριτήρια του χρήστη. Σε αυτό το στάδιο οι προτεινόμενες κατοικίες δεν έχουν ακόμη εμφανιστεί, επειδή η διαδικασία αναζήτησης βρίσκεται σε εξέλιξη.

Στην τελευταία εικόνα φαίνεται η ολοκλήρωση της συγκεκριμένης ροής με αποτέλεσμα να μην εντοπιστούν κατάλληλα ακίνητα. Το panel κατάστασης ενημερώνεται με την ένδειξη «ΔΕΝ ΒΡΕΘΗΚΑΝ ΑΚΙΝΗΤΑ» και εμφανίζεται σχετικό μήνυμα που ενημερώνει τον χρήστη ότι δεν βρέθηκε κατοικία που να ικανοποιεί τις απαιτήσεις του. Η περιοχή «Προτεινόμενες Κατοικίες» παραμένει κενή και δεν πραγματοποιείται το επόμενο στάδιο δημιουργίας τελικών προτάσεων. Με αυτόν τον τρόπο φαίνεται στην πράξη η δυναμική αλλαγή της κατάστασης εκτέλεσης και η σωστή διαχείριση της περίπτωσης όπου ένα έγκυρο αίτημα δεν επιστρέφει αποτελέσματα.

Περίπτωση επιτυχημένης εκτέλεσης

Περιγραφή Αναζήτησης

Ψάχνω κατοικία στη Σητεία με ενοίκιο έως 450 ευρώ και τουλάχιστον 1 υπνοδωμάτιο. Ιδανικά, να είναι επιπλωμένη και να έχει θέση στάθμευσης.

Υποχρεωτικά στοιχεία: μέγιστο ενοίκιο και ελάχιστος αριθμός υπνοδωματίων

Επεξεργασία...

ΚΑΤΑΣΤΑΣΗ ΕΚΤΕΛΕΣΗΣ

ΕΛΕΓΧΟΣ ΑΙΤΗΜΑΤΟΣ

Ελέγχεται το αίτημα που έχετε καταχωρήσει ως προς την εγκυρότητά του.

Προτεινόμενες Κατοικίες

Οι προτεινόμενες κατοικίες θα εμφανιστούν εδώ μετά την ολοκλήρωση της αναζήτησης.

Περιγραφή Αναζήτησης

Ψάχνω κατοικία στη Σητεία με ενοίκιο έως 450 ευρώ και τουλάχιστον 1 υπνοδωμάτιο. Ιδανικά, να είναι επιπλωμένη και να έχει θέση στάθμευσης.

Υποχρεωτικά στοιχεία: μέγιστο ενοίκιο και ελάχιστος αριθμός υπνοδωματίων

Επεξεργασία...

ΚΑΤΑΣΤΑΣΗ ΕΚΤΕΛΕΣΗΣ

ΑΝΑΖΗΤΗΣΗ ΑΚΙΝΗΤΩΝ

Έγκυρο Αίτημα. Πραγματοποιείται αναζήτηση ακινήτων βάσει των προτιμήσεων και απαιτήσεών σας.

Προτεινόμενες Κατοικίες

Οι προτεινόμενες κατοικίες θα εμφανιστούν εδώ μετά την ολοκλήρωση της αναζήτησης.

🔍 Περιγραφή Αναζήτησης

Ψάχνω κατοικία στη Σητεία με ενοίκιο έως 450 ευρώ και τουλάχιστον 1 υπνοδωμάτιο. Ιδανικά, να είναι επιπλωμένη και να έχει θέση στάθμευσης.

🕒 Υποχρεωτικά στοιχεία: μέγιστο ενοίκιο και ελάχιστος αριθμός υπνοδωματίων

Επεξεργασία...

🏠 Προτεινόμενες Κατοικίες

Οι προτεινόμενες κατοικίες θα εμφανιστούν εδώ μετά την ολοκλήρωση της αναζήτησης.

ΚΑΤΑΣΤΑΣΗ ΕΚΤΕΛΕΣΗΣ

ΔΗΜΙΟΥΡΓΙΑ ΠΡΟΤΑΣΕΩΝ

Βρέθηκαν διαθέσιμες κατοικίες που πληρούν τις βασικές απαιτήσεις σας. Τώρα δημιουργούνται προτάσεις για την καλύτερη δυνατή επιλογή.

🔍 Περιγραφή Αναζήτησης

Ψάχνω κατοικία στη Σητεία με ενοίκιο έως 450 ευρώ και τουλάχιστον 1 υπνοδωμάτιο. Ιδανικά, να είναι επιπλωμένη και να έχει θέση στάθμευσης.

🕒 Υποχρεωτικά στοιχεία: μέγιστο ενοίκιο και ελάχιστος αριθμός υπνοδωματίων

Αναζήτηση

🏠 Προτεινόμενες Κατοικίες

#1

Διαμέρισμα 45 τ.μ. στη Σητεία

400 €

✚ Σητεία 45 τ.μ. 1 υπνοδωμάτιο Επιπλωμένο

Πλήρως ανακαινισμένο και επιπλωμένο διαμέρισμα 45 τ.μ. στο κέντρο της Σητείας, σε υπερψωμένο ισόγειο, γωνιακό και πολύ φωτεινό. Διαθέτει 1 υπνοδωμάτιο, 1 μπάνιο, αυτόνομη θέρμανση πετρελαίου, κλιματισμό, μπαλκόνι, κοινόχρηστο κήπο και ανοιχτή θέση στάθμευσης. Παραδίδεται πλήρως εξοπλισμένο, με internet και έτοιμο για άμεση κατοίκηση.

Αξιολόγηση: Το διαμέρισμα φαίνεται να ταιριάζει πολύ καλά με τις απαιτήσεις σας, καθώς είναι επιπλωμένο, διαθέτει 1 υπνοδωμάτιο και έχει τιμή 400€, δηλαδή εντός του μέγιστου προϋπολογισμού σας. Επιπλέον, προσφέρει ανοιχτή θέση στάθμευσης, που ήταν βασική προϋπόθεση, ενώ είναι πλήρως ανακαινισμένο, πλήρως εξοπλισμένο και έτοιμο για άμεση κατοίκηση, κάτι που το καθιστά πρακτική επιλογή. Ως θετικά στοιχεία ξεχωρίζουν επίσης η φωτεινότητα, το μπαλκόνι, το internet και η κεντρική τοποθεσία στη Σητεία. Η βασική παραχώρηση είναι ότι πρόκειται για υπερψωμένο ισόγειο και όχι για μεγαλύτερο ή νεόδμητο χώρο, όμως συνολικά η πρόταση είναι πολύ ισορροπημένη και συμβατή με όσα ζητάτε.

Προβολή αγγελίας ↗

ΚΑΤΑΣΤΑΣΗ ΕΚΤΕΛΕΣΗΣ

ΔΗΜΙΟΥΡΓΗΘΗΚΑΝ ΤΕΛΙΚΕΣ ΠΡΟΤΑΣΕΙΣ

Έχουν δημιουργηθεί τελικές προτάσεις για τις κατοικίες που πληρούν όσο το δυνατόν περισσότερο τις απαιτήσεις σας.

🏠 Προτεινόμενες Κατοικίες

#1

Διαμέρισμα 45 τ.μ. στη Σητεία

400 €

✚ Σητεία 45 τ.μ. 1 υπνοδωμάτιο Επιπλωμένο

Πλήρως ανακαινισμένο και επιπλωμένο διαμέρισμα 45 τ.μ. στο κέντρο της Σητείας, σε υπερυψωμένο ισόγειο, γωνιακό και πολύ φωτεινό. Διαθέτει 1 υπνοδωμάτιο, 1 μπάνιο, αυτόνομη θέρμανση πετρελαίου, κλιματισμό, μπαλκόνι, κοινόχρηστο κήπο και ανοιχτή θέση στάθμευσης. Παραδίδεται πλήρως εξοπλισμένο, με internet και έτοιμο για άμεση κατοίκηση.

Αξιολόγηση: Το διαμέρισμα φαίνεται να ταιριάζει πολύ καλά με τις απαιτήσεις σας, καθώς είναι επιπλωμένο, διαθέτει 1 υπνοδωμάτιο και έχει τιμή 400€, δηλαδή εντός του μέγιστου προϋπολογισμού σας. Επιπλέον, προσφέρει ανοιχτή θέση στάθμευσης, που ήταν βασική προϋπόθεση, ενώ είναι πλήρως ανακαινισμένο, πλήρως εξοπλισμένο και έτοιμο για άμεση κατοίκηση, κάτι που το καθιστά πρακτική επιλογή. Ως θετικά στοιχεία ξεχωρίζουν επίσης η φωτεινότητα, το μπαλκόνι, το internet και η κεντρική τοποθεσία στη Σητεία. Η βασική παραχώρηση είναι ότι πρόκειται για υπερυψωμένο ισόγειο και όχι για μεγαλύτερο ή νεόδμητο χώρο, όμως συνολικά η πρόταση είναι πολύ ισορροπημένη και συμβατή με όσα ζητάτε.

[Προβολή αγγελίας ↗](#)

🏠 Προτεινόμενες Κατοικίες

#4

Διαμέρισμα 30 τ.μ.

400 €

✚ Σητεία 30 τ.μ. 1 υπνοδωμάτιο Επιπλωμένο

Ενοικιάζεται επιπλωμένο διαμέρισμα 30 τ.μ. στη Σητεία, 2ου ορόφου, με 1 υπνοδωμάτιο και 1 μπάνιο. Βρίσκεται σε οργανωμένο συγκρότημα, μόλις 5 λεπτά από την παραλία, και διαθέτει κλιματισμό, ηλεκτρικές συσκευές, κοινόχρηστο χώρο μπαρμπεκιού και θέση στάθμευσης.

Αξιολόγηση: Το ακίνητο φαίνεται αρκετά καλή επιλογή σε σχέση με τις απαιτήσεις σας, καθώς είναι επιπλωμένο, διαθέτει 1 υπνοδωμάτιο και έχει τιμή 400€, άρα βρίσκεται εντός του μέγιστου budget σας. Επιπλέον, περιλαμβάνει θέση στάθμευσης, κάτι που καλύπτει άμεσα ένα βασικό ζητούμενο, ενώ η τοποθεσία στη Σητεία και η εγγύτητα στην παραλία είναι θετικά στοιχεία για κάποιον που θέλει πρακτικότητα και εύκολη πρόσβαση. Από την άλλη, το μέγεθος των 30 τ.μ. είναι σχετικά μικρό, οπότε προσφέρει περιορισμένο χώρο, και βρίσκεται στον 2ο όροφο, κάτι που μπορεί να μην ταιριάζει σε όλους. Συνολικά, πρόκειται για μια λειτουργική και πλήρως συμβατή πρόταση με τα βασικά σας κριτήρια, με κύριο συμβιβασμό τον περιορισμένο χώρο.

[Προβολή αγγελίας ↗](#)

Στις εικόνες παρουσιάζεται αναλυτικά η πλήρης επιτυχής ροή εκτέλεσης του SitiaHome, από τη στιγμή που ο χρήστης υποβάλλει ένα έγκυρο αίτημα μέχρι την εμφάνιση των τελικών προτεινόμενων κατοικιών. Το prompt του χρήστη περιλαμβάνει τα δύο υποχρεωτικά στοιχεία, δηλαδή μέγιστο ενοίκιο έως 450 ευρώ και τουλάχιστον ένα υπνοδωμάτιο, ενώ παράλληλα δηλώνονται πρόσθετες προτιμήσεις, όπως η κατοικία να είναι επιπλωμένη και να διαθέτει θέση στάθμευσης. Αμέσως μετά την έναρξη της διαδικασίας, στο panel «Κατάσταση Εκτέλεσης» εμφανίζεται η ένδειξη «ΕΛΕΓΧΟΣ ΑΙΤΗΜΑΤΟΣ», η οποία ενημερώνει τον χρήστη ότι το σύστημα επεξεργάζεται το prompt και ελέγχει την εγκυρότητά του. Σε αυτό το στάδιο δεν έχουν ακόμη εμφανιστεί προτάσεις, καθώς η εφαρμογή περιμένει πρώτα να ολοκληρωθεί το Requirements Processing και το Requirements Validation.

Αφού το αίτημα χαρακτηριστεί έγκυρο, η εφαρμογή μεταβαίνει στο στάδιο «ΑΝΑΖΗΤΗΣΗ ΑΚΙΝΗΤΩΝ». Η αλλαγή αυτή στο panel της κατάστασης εκτέλεσης δείχνει ότι η ροή έχει περάσει από το validation στο Property Search component. Στο συγκεκριμένο στάδιο πραγματοποιείται αναζήτηση στην πλατφόρμα ΧΕ και εφαρμόζεται το αρχικό filtering με βάση τα βασικά κριτήρια του χρήστη, όπως το μέγιστο ενοίκιο και ο ελάχιστος αριθμός υπνοδωματίων. Η περιοχή των προτεινόμενων κατοικιών εξακολουθεί προσωρινά να παραμένει κενή, επειδή το σύστημα δεν έχει ακόμη ολοκληρώσει την αξιολόγηση και επιλογή των υποψήφιων αγγελιών. Όταν βρεθούν κατοικίες που ικανοποιούν τα βασικά κριτήρια, το panel αλλάζει ξανά και εμφανίζει την ένδειξη «ΔΗΜΙΟΥΡΓΙΑ ΠΡΟΤΑΣΕΩΝ», σηματοδοτώντας τη μετάβαση στο Recommendation / Ranking Engine.

Στο στάδιο της δημιουργίας προτάσεων, οι υποψήφιες αγγελίες επεξεργάζονται πιο αναλυτικά. Για κάθε URL ανακτώνται τα επιμέρους δεδομένα της αγγελίας, τα οποία οργανώνονται σε κοινό structured schema, και στη συνέχεια δημιουργείται μία ξεχωριστή LLM-based αξιολόγηση για κάθε κατοικία. Η αξιολόγηση αυτή δεν εξετάζει μόνο αν το ακίνητο ικανοποιεί τα βασικά φίλτρα, αλλά και το πώς συνδέεται με τις πρόσθετες προτιμήσεις του χρήστη. Αν οι διαθέσιμες κατοικίες είναι περισσότερες από πέντε, πραγματοποιείται επιπλέον στάδιο επιλογής και ranking, ώστε να επιστραφούν μόνο οι πέντε καταλληλότερες σε σειρά από την καλύτερη προς την ασθενέστερη αντιστοίχιση. Όταν η διαδικασία ολοκληρωθεί, η κατάσταση αλλάζει σε «ΔΗΜΙΟΥΡΓΗΘΗΚΑΝ ΤΕΛΙΚΕΣ ΠΡΟΤΑΣΕΙΣ», ενημερώνοντας ότι το σύστημα έχει πλέον ολοκληρώσει όλη τη backend επεξεργασία.

Στις επόμενες εικόνες φαίνεται η τελική μορφή των αποτελεσμάτων στο Frontend. Οι κατοικίες εμφανίζονται σε ξεχωριστές recommendation cards, καθεμία από τις οποίες περιλαμβάνει τη θέση της στη λίστα, τον τίτλο της αγγελίας, την τιμή, την περιοχή, τα τετραγωνικά μέτρα, τον αριθμό υπνοδωματίων και την κατάσταση επίπλωσης. Επιπλέον, παρουσιάζεται σύντομη περιγραφή του ακινήτου και σύνδεσμος «Προβολή αγγελίας», μέσω του οποίου ο χρήστης μπορεί να μεταβεί στην αρχική καταχώριση της ΧΕ. Η ύπαρξη αυτών των structured πληροφοριών επιτρέπει στον χρήστη να συγκρίνει γρήγορα τις βασικές ιδιότητες των προτεινόμενων κατοικιών χωρίς να χρειάζεται να ανοίξει άμεσα κάθε αγγελία ξεχωριστά.

Ιδιαίτερη σημασία έχει το πεδίο «Αξιολόγηση» που εμφανίζεται μέσα σε κάθε recommendation card. Η αξιολόγηση παράγεται από το LLM με βάση τόσο τα χαρακτηριστικά της εκάστοτε κατοικίας όσο και τις απαιτήσεις του χρήστη. Στα παραδείγματα που φαίνονται στις εικόνες, το σύστημα αναγνωρίζει ότι οι κατοικίες είναι επιπλωμένες, διαθέτουν ένα υπνοδωμάτιο και βρίσκονται εντός του μέγιστου budget των 450 ευρώ, επομένως ικανοποιούν τα βασικά κριτήρια. Παράλληλα, σχολιάζονται επιπλέον στοιχεία, όπως το μέγεθος του ακινήτου, η τοποθεσία, η κατάσταση επίπλωσης και η ύπαρξη ή μη σαφούς πληροφορίας για θέση στάθμευσης. Με αυτόν τον τρόπο, η αξιολόγηση δεν λειτουργεί ως απλό θετικό σχόλιο, αλλά παρουσιάζει και πιθανούς περιορισμούς ή trade-offs, ώστε ο χρήστης να έχει πιο ολοκληρωμένη εικόνα της κάθε πρότασης.

3. Συμπεράσματα και Μελλοντικές Επεκτάσεις

3.1 Συμπεράσματα

Το SitiaHome ολοκληρώθηκε ως ένα λειτουργικό Software/AI Engineering MVP που καλύπτει την πλήρη βασική ροή εύρεσης και πρότασης κατοικιών για μακροχρόνια ενοικίαση στον Δήμο Σητείας. Η εφαρμογή επιτρέπει στον χρήστη να εκφράζει τις ανάγκες του σε φυσική γλώσσα, μετατρέπει το αίτημα σε δομημένες απαιτήσεις, ελέγχει την εγκυρότητά του, αναζητά σχετικές αγγελίες στην πλατφόρμα ΧΕ και εφαρμόζει αρχικό filtering με βάση τα βασικά κριτήρια. Στη συνέχεια, οι υποψήφιες κατοικίες επεξεργάζονται πιο αναλυτικά, οργανώνονται σε κοινό structured schema, αξιολογούνται με χρήση LLM και, όταν απαιτείται, επιλέγονται και κατατάσσονται οι καταλληλότερες προτάσεις πριν παρουσιαστούν στον χρήστη μέσω του Frontend.

Μέσα από την υλοποίηση του project επιτεύχθηκε ο συνδυασμός διαφορετικών τεχνολογιών και τεχνικών σε μία ενιαία λειτουργική εφαρμογή. Το React και το Vite χρησιμοποιήθηκαν για το Frontend, το FastAPI και ο Unicorn για το Backend/API, ενώ οι HTTP requests, η εξαγωγή structured δεδομένων από HTML, η χρήση Pydantic models και η ενσωμάτωση Large Language Models αποτέλεσαν βασικά στοιχεία της συνολικής αρχιτεκτονικής. Ιδιαίτερη σημασία είχε ο διαχωρισμός της εφαρμογής σε επιμέρους components με σαφείς αρμοδιότητες, ώστε η επεξεργασία απαιτήσεων, το validation, η αναζήτηση ακινήτων και το recommendation/ranking να υλοποιούνται ανεξάρτητα αλλά να συνεργάζονται μέσα από μια κοινή ροή. Με αυτόν τον τρόπο, το project ανέδειξε στην πράξη βασικές αρχές Software Engineering, όπως modularity, separation of concerns και οργανωμένη επικοινωνία μεταξύ διαφορετικών τμημάτων του συστήματος.

Συνολικά, ο βασικός στόχος του project επιτεύχθηκε, καθώς δημιουργήθηκε ένα λειτουργικό prototype που δείχνει πώς τεχνικές AI και data extraction μπορούν να ενσωματωθούν σε μία πραγματική εφαρμογή για την υποστήριξη μιας καθημερινής διαδικασίας. Παρότι το SitiaHome δεν σχεδιάστηκε ως production-ready εμπορικό σύστημα, η παρούσα υλοποίηση αποτελεί μια ολοκληρωμένη βάση πάνω στην οποία μπορούν να προστεθούν μελλοντικά επιπλέον δυνατότητες. Παράλληλα, το project αποτέλεσε πρακτική εφαρμογή τόσο της οργανωμένης διαδικασίας ανάπτυξης λογισμικού όσο και της αξιοποίησης σύγχρονων AI εργαλείων για την κατανόηση φυσικής γλώσσας, την οργάνωση δεδομένων και την παραγωγή πιο χρήσιμων και εξατομικευμένων αποτελεσμάτων.

3.2 Μελλοντικές Επεκτάσεις

A. Υποστήριξη περισσότερων πηγών αγγελιών

Μία σημαντική μελλοντική επέκταση θα ήταν η ενσωμάτωση περισσότερων πλατφορμών αγγελιών πέρα από τη Χρυσή Ευκαιρία. Με αυτόν τον τρόπο το σύστημα θα μπορούσε να αναζητά ταυτόχρονα σε περισσότερες πηγές, αυξάνοντας τον αριθμό των διαθέσιμων κατοικιών και προσφέροντας πιο ολοκληρωμένα αποτελέσματα στον χρήστη.

B. Μόνιμη αποθήκευση δεδομένων και User Profiles

Θα μπορούσε να προστεθεί βάση δεδομένων για την αποθήκευση ιστορικού αναζητήσεων, αγαπημένων ακινήτων και προηγούμενων επιλογών. Σε συνδυασμό με user accounts, το σύστημα θα μπορούσε να δημιουργεί προφίλ προτιμήσεων και να προσφέρει περισσότερο προσωποποιημένες προτάσεις σε επόμενες αναζητήσεις.

Γ. Βελτιστοποίηση Απόδοσης και Μείωση Κόστους

Μελλοντικά θα μπορούσαν να προστεθούν τεχνικές όπως caching, περιορισμός επαναλαμβανόμενων HTTP requests και πιο στοχευμένη χρήση των LLM calls. Αυτό θα μπορούσε να μειώσει τον χρόνο απόκρισης, το API cost και την πιθανότητα εμφάνισης rate limits, ιδιαίτερα όταν αυξηθεί ο αριθμός των διαθέσιμων αγγελιών.

Δ. Production Deployment και Πρόσθετες Λειτουργίες

Σε μία πιο ολοκληρωμένη έκδοση, το SitiaHome θα μπορούσε να γίνει deploy σε production περιβάλλον και να υποστηρίζει authentication, monitoring και πιο ολοκληρωμένο error handling.

4. Αναφορές

1. <https://www.ibm.com/think/topics/sdlc>, IBM, “What is the software development life cycle (SDLC)?”
2. <https://react.dev/learn>, Meta, “React Documentation”.
3. <https://fastapi.tiangolo.com/>, Sebastián Ramírez, “FastAPI Documentation”.
4. <https://www.xe.gr/>
5. <https://www.crummy.com/software/BeautifulSoup/bs4/doc/>, “Beautiful Soup Documentation”.
6. <https://uvicorn.dev/>, “Uvicorn documentation”.
7. <https://vite.dev/guide/>, “Vite documentation”.